Runtime Model Switching on the Edge Device

What LearnPerk switches during a journey, on which metrics, to which model, and why it is more efficient

Document	Model-switching specification and scenario set — the follow-on document named at the end of *LearnPerk Edge Data & Model Switching v3*
Scope	Edge only. The commuter's mobile device, during transit, at runtime. Cloud aggregation and cloud-side switching are explicitly out of scope.
Status	Design specification. Proposes an architecture, a metric set, a model registry and an evaluation protocol. Not yet implemented.
Builds on	EdgeMLBalancer (arXiv:2502.06493) · HarmonE (arXiv:2505.13693) · CALM (arXiv:2602.03632) · the EdgeMLBalancer Android reference implementation
Grounded in	LearnPerk Data Dictionary (Sept 2026) and the four combined files: 9,990 offer impressions, 19,334 content views, 2,000 commuters, 70,758 events
Changes	Section 8 specifies the additions this design forces on the LearnPerk Data Dictionary — three new edge files, one reference table, and new columns on all four existing files
Version	v1.0 · 25 September 2026 · Internal

In one paragraph

A commuter's phone is not a stable computer. Across a single twelve-minute metro leg the network drops out, the battery falls, the carriage fills, the train runs late, and the window in which anything can usefully be shown shrinks to nothing. LearnPerk today would answer all of that with one model per task, sized for the best case. This document proposes the alternative: a small registry of models per task, spanning a deliberate accuracy/cost spectrum, and a MAPE-K loop on the device that picks between them every five seconds on measured evidence — CPU, battery, thermal state, latency, connectivity, confidence, drift, and how much of the journey is left. The loop switches down to stay alive, switches up when the moment is worth it, and is judged on whether it keeps acceptance while spending less.

Contents

1 Why LearnPerk needs runtime switching at all
1.1 The evidence in our own data
1.2 What a single fixed model costs us
1.3 What we take from each of the three papers

2 The adaptation architecture
2.1 Managed system, managing system, decision map
2.2 The MAPE-K loop on device
2.3 Cadence, dwell time and the cost of switching

3 The metrics — what we measure and why
3.1 Group R: resource and device
3.2 Group Q: model quality
3.3 Group C: journey and context
3.4 Sampling, aggregation and smoothing

4 The switching decision
4.1 Feasibility gate
4.2 Utility, cost and the composite score
4.3 Dynamic boundaries
4.4 Exploration, hysteresis and rollback
4.5 Operating modes
4.6 The decision table

5 The model registry — what models, named
5.1 Five task families
5.2 Task A: journey-state classifier
5.3 Task B: offer ranking
5.4 Task C: content ranking
5.5 Task D: incentive policy
5.6 Task E: the natural-language layer
5.7 Residency, memory budget and caching

6 What data each model needs
6.1 Feature sets by task
6.2 Coverage, nulls and graceful degradation

7 Five use cases
7.1 UC-1 Lisa — the tunnel
7.2 UC-2 John — the failing battery
7.3 UC-3 Sanne — the delayed, crowded train
7.4 UC-4 Daan — the commuter we know nothing about
7.5 UC-5 Priya — four minutes to the interchange
7.6 Cross-case summary

8 What this changes in the Data Dictionary
8.1 Three new edge files
8.2 One new reference table
8.3 New columns on the four existing files
8.4 Retention, consent and privacy

9 How we test the switching
9.1 Research questions
9.2 Baselines
9.3 Replay harness and off-policy evaluation
9.4 Evaluation metrics
9.5 Per-use-case acceptance tests
9.6 On-device field trial

10 Threats to validity and open questions

11 Appendix — parameter defaults

1 Why LearnPerk needs runtime switching at all

Model switching is easy to argue for in the abstract and hard to justify in the particular. The abstract argument — edge devices are constrained, therefore adapt — is true of every phone application ever written and tells us nothing about LearnPerk. This section makes the particular argument, from the dataset we already hold.

1.1 The evidence in our own data

Every figure below is computed from `combined_offers.csv` (9,990 impressions) and `combined_content_views.csv` (19,334 views). They are not illustrative. They are the reason this document exists.

#	Observation	Measured value	What it forces
1	A metro leg is mostly not online	Of metro-leg impressions, only **41.6%** are `Online`. **35.6%** are Degraded and **22.9%** are fully Offline. Tram, bus, rail and micromobility all sit near 90% online; metro is the outlier.	Any model whose serving path touches the network is unavailable for the majority of metro time. Amsterdam's commuter core is metro. An offline-capable tier is not a fallback, it is the common case.
2	Content stops when the network does	Content views split 15,025 Online / 4,309 Degraded / **0 Offline**. Offers still appear offline (1,000 impressions); content never does.	We have a hard capability hole, not a quality dip. An on-device content ranker over a cached item pool converts zero into something.
3	The window is short, sometimes very short	Leg duration: median **11.9 min**, 10th percentile **8.1 min**, and **8.6%** of legs under 8 minutes. Most commuters board mid-route, which shortens it further.	A ranker whose p95 latency is 40 ms is irrelevant on a 20-minute leg and decisive on a four-minute one. Remaining window must be a first-class switching metric, not a UI concern.
4	One impression in five is on a low battery	**19.8%** of impressions occur below 30% charge, **7.7%** below 20%. `truth_battery_anxiety` sets a personal disengagement threshold between 20% and 40%, per commuter.	Battery is not a uniform constraint. The trigger point is personal, so the threshold must be per-user rather than a global constant.
5	Most of the fleet is not our dev phone	Device mix is **30% Low-tier**, 45% Mid-tier, 25% High-tier. **5.8%** of impressions are low-tier *and* under 30% battery at the same moment.	One model sized for a high-tier device is mis-sized for three quarters of the fleet. Sizing for low-tier throws capability away on the other 70%.
6	Features go missing, predictably	`crowding_level` is null on **25%** of legs by design — GVB does not publish occupancy everywhere. `accept_rate_before` is null on **17%** of impressions.	Model choice must be conditioned on feature coverage. A model that needs crowding is the wrong model on a quarter of legs, however good it is elsewhere.
7	Cold start is not an edge case	Median tenure at impression is **169 days**, but the 10th percentile is **26 days** and 25% of impressions have **one or fewer** prior offers seen. Cache is `empty`, `stale` or `evicted` on **23%** of impressions.	The sparse-signal path runs often enough to deserve its own model, rather than a degraded call into the rich-history one.
8	Peak is a different operating regime	Delay above five minutes: **12.6%** at peak against **5.9%** off-peak. High crowding: **25.1%** at peak against **9.2%** off-peak.	Disruption clusters. The conditions that make a journey worth intervening in are the same conditions that load device and network hardest.

Table 1.1 — Eight measured facts about LearnPerk journeys, each of which a single fixed model handles badly. Percentages are over impressions unless stated otherwise.

1.2 What a single fixed model costs us

Put those eight together and the shape of the problem is clear. Whatever single model we pick, we pick it for one operating point and then live with it at every other. Three failure modes follow, and each is a design failure rather than a tuning failure.

- **Size for the best case** and the model is unserveable on a low-tier phone in a tunnel at 18% battery — which, from rows 1, 4 and 5 of Table 1.1, describes a real and recurring slice of sessions. The commuter sees nothing, and we log nothing, so the failure never appears in the metrics that would tell us about it.
- **Size for the worst case** and we leave accuracy on the table for the other 70% of sessions. A commuter on a high-tier phone, on a 25-minute delayed tram at 74% battery, is the most valuable moment LearnPerk gets; we would serve it with the model we built for the tunnel.

- **Size for the average** and we get the worst of both: too heavy to survive the bad case, too light to exploit the good one. The average journey does not exist. The dictionary says so plainly — the departure-hour distribution is twin-peaked, and averaging it lands near midday, when almost nobody travels.

Switching is the answer to a problem that is about *variance*, not about average resource pressure. The device's operating point moves further within one twelve-minute leg than it does between two different commuters.

1.3 What we take from each of the three papers

This design is not novel for its own sake. Each mechanism below is taken from a published approach that has already been evaluated, then adapted where LearnPerk's problem genuinely differs. Where we depart, the departure is stated and argued.

Source	Mechanism we adopt	How LearnPerk adapts it
EdgeMLBalancer arXiv:2502.06493 Matathammal et al., 2025	A spectrum of four TFLite detectors on one Android device; a MAPE-K loop running at 1 Hz; CPU read from `/proc/[pid]/stat`; a per-model score combining instantaneous and running-average cost against confidence; and ε-**greedy selection at** ε = **0.1** so no model starves. Reported: 17.36% mean accuracy at 19.90% CPU; fairness improved 43.6% over a naive threshold policy and 41.5% over round-robin; 0.85 s mean switch time; 2.10 mAh against 5.25 mAh for naive.	We keep the spectrum, the 1 Hz monitor, the ε-greedy fairness floor and the reference implementation's CPU probe. We **replace the score**: their $S = \min(U, U_{avg}) \cdot (1 - C_{avg}/C)$ is minimised over a single cost channel (CPU) against a single quality channel (per-frame detection confidence). LearnPerk has four cost channels and no per-frame ground truth, so we move to an explicit weighted utility-minus-cost form (§4.2). We also **add dwell-time hysteresis**: re-deciding at 1 Hz with a 0.85 s switch cost would let a twelve-minute leg spend a large share of itself mid-switch (§2.3).
HarmonE arXiv:2505.13693 Bhatt et al., 2025	A design-time **Decision Map** binding sustainability goals to adaptation boundaries; a blended score $S_i = \alpha A_i + (1-\alpha)(1-E_{avg,i})$; **EMA smoothing** so transient spikes do not trigger adaptation; a **control-theoretic dynamic threshold** $\theta_{i+1} = \theta_i + \gamma(E_{avg} - ref - E_{avg,i})$ that tightens under sustained overspend and loosens for transient ones; drift detection against a reference distribution; and a versioned model repository so a past model can be *reused* rather than retrained. Reported: 54.5% less energy than LSTM with periodic retraining, at 95% of its accuracy and 53.4% better inference time.	We adopt the Decision Map as our **operating modes** (§4.5), and the EMA, the dynamic boundary and the drift trigger essentially unchanged. We **drop the retraining arm**: retraining on a commuter's phone mid-journey is not credible, so a drift alarm at the edge switches to a different pre-trained model and raises a flag for the cloud instead. HarmonE's versioned repository becomes a small set of **regime-specific models** — peak, off-peak, disrupted — trained centrally and shipped ahead of time.
CALM arXiv:2602.03632 Jain et al., 2026	A fleet of domain-specialised SLMs in place of one general LLM; a routing score combining **static semantic similarity** (cosine between the query and each model's description) with **rolling runtime metrics** under a tunable weight `k`; pro-latency, pro-energy, pro-confidence and pro-cache routing variants; an **LRU cache of models resident in memory** with the rest on disk; and FIFO scheduling chosen over STCF because STCF starves the slow models. Reported: 40–50% lower latency and 50–60% lower energy than single-LLM baselines, confidence 15–30% higher, and a cache size of one cutting peak VRAM from 32.5 GB to 7.9 GB.	CALM is the direct template for our natural-language layer (Task E, §5.6) — GVB bulletins, free-text preferences and mid-journey Q&A. We take the similarity-plus-metrics routing score and the **cache-residency** mechanism nearly verbatim, because a phone has far less room than CALM's 48 GB A6000: our cache size is **one** resident language model, so the eviction decision matters more, not less. We **invert the deployment assumption**: CALM holds every model on one server and routes across them, whereas we keep at most one language model warm and treat a cold start as a switching cost the score must pay for (§5.7).

Table 1.2 — Provenance of each mechanism and where LearnPerk departs from it. The reported figures are the original authors', measured on their own workloads; they motivate this design, they do not predict our results.

> **The design claim in one sentence**
>
> Take EdgeMLBalancer's per-device model spectrum and ε-greedy fairness floor, govern it with HarmonE's design-time goals and self-tightening boundaries, route the language tier with CALM's similarity-plus-metrics score under a cache of one — and then add the two things none of the three needed: a **remaining-journey-window** term, because the commuter gets off the train, and **dwell-time hysteresis**, because switching is not free.

2 The adaptation architecture

2.1 Managed system, managing system, decision map

The architecture is HarmonE's three-part split, instantiated on a phone. The **managed system** is LearnPerk as the commuter experiences it: the offer surface, the content feed, the assistant, and the inference calls behind them. The **managing system** is a background service that watches the managed system and its environment and changes which model each surface calls. The **decision map** is a design-time artefact — a table, not code — in which we write down what LearnPerk is trying to be good at and what we will accept losing to stay within budget. Everything the managing system does at runtime traces to a boundary written in the decision map.

Layer	Contents	Lives where
Managed system	Offer surface, content feed, in-journey assistant, journey-state inference. Each surface calls its task through a stable interface and does not know which model is behind it.	App process, foreground
Managing system	Monitor, Analyser, Planner, Executor and the Knowledge store. One foreground service, one worker thread, one 5 s tick.	App process, background thread
Knowledge	Model registry, score table, operating-mode definitions, rolling metric windows, the per-user battery threshold, the drift reference distribution, and the switch log.	On-device store, synced to cloud on Wi-Fi
Decision map	The goals and their boundaries: acceptance retention, energy per hour, p95 latency by journey stage, memory ceiling, offline availability, explainability. Authored by us, versioned, shipped with the app.	Config, shipped and remotely tunable

Table 2.1 — The four layers. Only the decision map is authored by hand; the rest is mechanism.

The decision map matters more than it looks. HarmonE's central point is that adaptation without a stated goal degenerates into reacting to whatever metric moved last. Writing the goals down first is what lets us say, later, that a switch was *correct* rather than merely that it happened. LearnPerk's six goals and their initial boundaries are in Table 2.2; each boundary is the number the analyser compares against.

Goal	Dimension	Boundary (initial)	Which metric tests it
Keep the commercial result	Technical / economic	Offer acceptance rate within **2 percentage points** of the always-heaviest configuration, measured over a rolling 200 impressions	Realised accept rate A_{avg}, compared off-policy against the heavy arm
Spend less energy	Environmental	Median ≤ **18 mAh per commuting hour** attributable to LearnPerk inference; reference level set from a two-week baseline	Battery drain rate ΔB and estimated energy per call E
Stay responsive	Technical / social	p95 end-to-end surface latency ≤ **120 ms** in normal stages, ≤ **60 ms** when the remaining window is under 120 s	Inference latency L, p50 and p95 over the rolling window
Never go dark	Social	≥ **99%** of legs receive at least one serveable recommendation, including offline legs and legs on low-tier devices	Serve-failure rate; count of ticks with no feasible model
Fit in the phone	Technical	Peak resident set attributable to models ≤ **280 MB** on mid-tier, ≤ **160 MB** on low-tier	Resident memory R; cache residency state
Stay explainable	Social	Every surfaced item carries the model that produced it and, where a switch preceded it, the trigger that caused the switch	`serving_model_id` and `switch_id` on every impression (§8)

Table 2.2 — LearnPerk's decision map, reduced to six goals and six boundaries. These are the numbers §4 compares against. They are deliberately conservative for v1 and are expected to move once the field trial in §9.6 reports.

2.2 The MAPE-K loop on device

The loop is the standard five components. What follows is what each one does in LearnPerk specifically, and what it hands to the next.

Phase	What it does in LearnPerk	Output
Monitor 1 Hz sample, 5 s aggregate	Reads three metric groups (§3). Resource metrics come from the platform: CPU from `/proc/self/stat` deltas as in the EdgeMLBalancer reference code, memory from `Debug.MemoryInfo`, charge and drain from `BatteryManager`, thermal state from `PowerManager.getCurrentThermalStatus()`. Quality metrics come from the inference wrapper, which times every call and records the top-1 calibrated score. Context metrics come from the GVB feed, the connectivity manager and the activity-recognition API.	A metric tuple per 5 s tick, written to the rolling window and to `edge_adaptation_state`
Analyse	Cleans and aggregates over the window; recomputes EMA scores for every model in every task spectrum; evaluates each decision-map boundary; updates the dynamic energy threshold; runs the drift test on the feature distribution. Raises a typed uncertainty when a boundary is crossed: `RESOURCE_PRESSURE`, `CONNECTIVITY_LOSS`, `WINDOW_CLOSING`, `QUALITY_DEGRADATION`, `COVERAGE_LOSS`, `DRIFT`, or `OPPORTUNITY`.	Zero or more typed uncertainties, plus a refreshed score table
Plan	Maps the uncertainty to a tactic, then selects a model. The feasibility gate removes candidates that cannot run at all (§4.1); the composite score ranks what remains (§4.2); ε-greedy exploration and dwell-time hysteresis decide whether to act on the ranking (§4.4). For Task E the score additionally carries CALM's semantic similarity term.	A target model per affected task, or the decision to hold
Execute	Performs the switch: loads the target interpreter, warms it with one synthetic call, atomically swaps the task binding, then releases the outgoing model unless it is cache-resident. Writes the switch event. If the warm-up call fails or exceeds its latency budget, rolls back and blacklists the target for 90 s.	New active binding; one row in `edge_switch_events`
Knowledge	Holds the model registry, the score table with per-model EMA, the operating-mode table, the per-user battery threshold learned from past disengagement, the drift reference distribution shipped with the model, and the log repository. Everything the other four phases read or write passes through here.	Durable on-device state, uploaded on Wi-Fi

Table 2.3 — The MAPE-K loop as instantiated on the commuter's device. The typed uncertainties in the Analyse row are the vocabulary the rest of this document uses.

2.3 Cadence, dwell time and the cost of switching

EdgeMLBalancer re-decides once per second and reports a mean switch time of 0.85 s. On a continuous camera feed that is acceptable, because the next frame is 16 ms away and a missed frame costs nothing. LearnPerk is different in two ways that matter, and copying the cadence unchanged would be a mistake.

- **Our decisions are sparse.** A commuter is shown a handful of offers and a handful of content items per leg, not 1,800 frames. Re-ranking at 1 Hz computes answers nobody asked for. The *monitor* should sample at 1 Hz, because resource signals are noisy and need averaging, but the *planner* should run at 5 s and the *executor* should act less often still.
- **Our switches are expensive relative to the work.** Loading a quantised gradient-boosted ranker costs single-digit milliseconds, but loading a language model costs hundreds of milliseconds to seconds and tens to hundreds of megabytes. A switch that saves 3 mJ of inference and costs 400 mJ of loading is a loss the score must be able to see.

We therefore separate three cadences and put an explicit price on switching.

Parameter	Symbol	Default	Rationale
Monitor sample period	—	1 s	Matches the EdgeMLBalancer reference implementation. CPU deltas below this are dominated by sampling noise.
Analyse / plan period	—	5 s	Roughly 140 ticks on a median 11.9-minute leg — enough to react inside a leg, few enough that the loop itself is not the load.
Rolling window	n	12 ticks (60 s)	One minute of history survives a tunnel and a station stop without being so long that a genuine regime change is smoothed away.
EMA smoothing factor	β	0.30	HarmonE's role for this parameter: suppress the spike, keep the trend. At 0.30 a step change is ~90% absorbed in six ticks, or 30 s.
Minimum dwell time	τ	20 s (light tasks) 120 s (Task E)	A model must hold the binding this long before it can be replaced by score alone. Prevents the thrash a 5 s planner would otherwise produce.
Switch margin	δ	0.05	The challenger's EMA must beat the incumbent's by this much. Below the margin the difference is not distinguishable from window noise.
Switch cost charge	κ	Measured, per model pair	The load-and-warm cost is amortised over the expected remaining window and *subtracted from the challenger's score*. This is why a heavy model is correctly rejected at minute eleven of a twelve-minute leg.

Parameter	Symbol	Default	Rationale
Exploration rate	ε	0.10, annealed to 0.02	EdgeMLBalancer's value. Annealed once a model has 500 logged calls on this device, because the fairness argument for exploring weakens as the estimates firm up.
Forced-switch bypass	—	Immediate	A feasibility-gate failure — network gone, thermal throttle, memory pressure — ignores dwell time and margin entirely. Gates are not negotiable.

Table 2.4 — Timing and hysteresis parameters. The switch-cost charge κ and the dwell time τ are the two additions relative to EdgeMLBalancer, and both exist because a journey ends.

Why the switch cost has to be in the score, not in a rule

It is tempting to handle switching cost with a rule — say, no heavy-model switches in the last three minutes of a leg. That rule is wrong at both ends: it forbids a switch that would genuinely pay off on a long delayed leg, and it permits one that will not pay off on a short one. Charging κ amortised over the *expected remaining window* gets both right from one mechanism, and it degrades sensibly when the window estimate is poor. The window estimate comes from `duration_minutes` for the leg less elapsed time, corrected by live `delay_minutes`.

3 The metrics — what we measure and why

This is the section the rest of the document depends on. A switching decision is only as defensible as the evidence behind it, and the evidence is these metrics. Each one below is given with its symbol, how it is obtained, how often, what it costs to read, and — the part that matters — **which switching decision it can actually justify**. A metric that cannot change a decision does not belong in the loop, however interesting it is.

The metrics fall into three groups. **Group R** is resource and device state: what the phone can afford. This is EdgeMLBalancer's territory, extended past CPU. **Group Q** is model quality: whether the model we are running is still any good. This is HarmonE's territory. **Group C** is journey and context: what the situation is worth and how long it will last. This group has no equivalent in any of the three papers, because none of them had a user who gets off a train.

3.1 Group R: resource and device

Sym	Metric	Source and cadence	Decision it justifies
U	CPU utilisation, process-attributed, %	Delta of `utime+stime` from `/proc/self/stat` over a 100 ms interval, clamped to [0,100]. Exactly the probe in the EdgeMLBalancer reference implementation. 1 Hz. Cost < 0.5 ms.	The primary **switch-down** signal for compute-bound tasks. Sustained `U` above the mode ceiling moves Task B and Task C down one tier.
R	Resident memory attributable to models, MB	`Debug.MemoryInfo.getTotalPss()` minus a measured baseline, plus the registry's declared footprint for each loaded model. 5 s.	A **hard gate**, not a score term. If loading a candidate would exceed the mode ceiling (Table 2.2), the candidate is infeasible regardless of how good it is. Also drives cache eviction in Task E.
B	Battery charge, %	`BatteryManager.BATTERY_PROPERTY_CAPACITY`. 5 s. Free.	Crosses a **per-user** threshold learned from that commuter's own past disengagement, in the 20–40% band the dictionary documents. Triggers the step down to Frugal, then Survival mode.
ΔB	Battery drain rate, %/hour	Least-squares slope of `B` over the last 10 minutes, attributed to LearnPerk by differencing against screen-off idle. 60 s.	Tests the environmental boundary directly. Feeds the **dynamic threshold** of §4.3: sustained overspend tightens the boundary, a one-off spike does not.
E	Estimated energy per inference, mJ	CPU-time of the call multiplied by a per-device-class power coefficient calibrated once per device tier, plus a fixed cost for a delegate hand-off. Per call.	The cost channel that makes models comparable across tiers. Replaces the crude `battery × CPU/100` proxy the EdgeMLBalancer code uses, which conflates a level with a rate.
T	Thermal status, 0–6	`PowerManager.getCurrentThermalStatus()`. 5 s. Free.	A **hard gate**. At `MODERATE` (2) all tier-3 and above models become infeasible; at `SEVERE` (3) only tier-0 and tier-1 survive. Thermal throttling makes latency estimates worthless, so we stop trusting the score and fall back to the gate.
L	Inference latency, ms (p50 and p95)	Wall clock around the inference wrapper, kept as a reservoir per model. Per call.	The responsiveness boundary. p95 — not the mean — is compared against the stage budget, because a commuter notices the slow call, not the average call.
D	Device performance class	`device_performance_class`, resolved once at install from SoC, core count and RAM. Static.	Selects which **column** of the model registry applies. A registry latency figure is meaningless without it: the fleet is 30/45/25 low/mid/high.
Γ	Delegate availability	Whether NNAPI, GPU or an NPU delegate initialises successfully, probed once per session and after any failure. Session-scoped.	A **hard gate** on the neural tiers. A model that needs a delegate and cannot get one is not a slow model, it is an absent one.

Table 3.1 — Group R. Note that three of the nine (R, T, Γ) act as gates rather than score terms. Gates are checked first and cannot be traded away, which is what stops the loop from optimising itself into a crash.

3.2 Group Q: model quality

The hard problem in this group is that LearnPerk has no per-call ground truth. EdgeMLBalancer could read a detection confidence off every frame; HarmonE could compute an R^2 against an observed traffic count within minutes. We learn whether an offer was good only when the commuter accepts or ignores it, and `latency_minutes` in the dictionary is bimodal — most act within minutes, a second group redeems just before expiry, with a maximum of 1,419 minutes. So the feedback for some decisions arrives a day later, on a different leg, possibly on a different network.

We therefore split Group Q into an **immediate** proxy available at inference time and a **delayed** truth used to correct the proxy offline. The loop switches on the proxy and is audited on the truth.

Sym	Metric	Availability	Decision it justifies
C	Top-1 calibrated confidence — the model's own probability for the item it ranked first	**Immediate.** Per call.	The direct analogue of EdgeMLBalancer's per-frame confidence, and the main quality term in the score. Must be *calibrated* per model, or a tier that is merely more confident beats a tier that is more correct.
M	Ranking margin — the gap between the top-1 and top-2 scores	**Immediate.** Per call.	Detects the case a raw confidence hides: the model is sure of its number but cannot separate the candidates. A collapsed margin over the window is evidence that a **heavier** model is warranted, not a lighter one.
Φ	Feature coverage — share of the model's required features that are non-null for this call	**Immediate.** Per call.	A **gate plus a penalty**. Below the model's declared minimum coverage it is infeasible; above it, a shortfall discounts the score proportionally. This is how the 25% of legs with no `crowding_level` and the 17% with no `accept_rate_before` are handled as first-class cases.
A_{avg}	Realised acceptance rate over a rolling 200 impressions, per model	**Delayed** by minutes to a day.	The truth the proxy is audited against. Enters the live score only through a slow-moving per-model prior, never as a fast signal, because reacting to it inside a leg would be reacting to yesterday.
ECE	Expected calibration error over the rolling window	**Delayed.** Recomputed nightly.	If a model's confidence stops meaning what it claims, `C` becomes unusable for that model. A high ECE demotes the model in the registry until the cloud ships a recalibrated version.
Ψ	Feature drift — PSI between the live feature distribution and the reference shipped with the model	**Immediate** at window granularity.	HarmonE's drift trigger. `Ψ > 0.25` on any top-10 feature raises `DRIFT`. At the edge this **switches to the regime-specific model** whose reference distribution is closest — it never retrains — and flags the cloud.
Ω	Serve-failure count — calls that timed out, threw, or returned an empty candidate set	**Immediate.** Per call.	Two failures inside one window blacklist the model for 90 s and force a switch down, bypassing dwell time. A model that does not answer is worse than a weaker model that does.

Table 3.2 — Group Q. The immediate/delayed split is the honest response to LearnPerk having no per-call ground truth; pretending otherwise would put a day-old signal inside a five-second loop.

3.3 Group C: journey and context

This group is what makes the document LearnPerk-specific rather than a restatement of EdgeMLBalancer. A traffic camera runs until someone turns it off. A commuter's session has a known, short, measurable end, and the value of the session is not uniform across it. Both facts belong in the switching decision.

Sym	Metric	Source	Decision it justifies
N	Connectivity state: Online / Degraded / Offline	`ConnectivityManager` plus an active reachability probe; `connectivity_state`. 5 s.	The single most consequential gate we have. Offline makes every cloud-assisted tier infeasible instantly, with no dwell-time grace. On metro legs this fires on **22.9%** of impressions, and Degraded on a further 35.6%.
W	Remaining window, seconds	Leg `duration_minutes` less elapsed, corrected by live `delay_minutes` and by `journey_stage`. 5 s.	Two roles. It sets the **latency budget** (tight when W is small), and it is the **denominator that amortises the switch cost** §2.3. With a median leg of 11.9 minutes and 8.6% under 8 minutes, this is not a rare consideration.
S	Journey stage: waiting / boarding / in-transit / approaching / interchange / alighting	Derived from vehicle position, motion state and the GVB feed. 5 s.	Selects the latency budget and the appropriate interaction. `interchange` is the high-value, low-window stage; `in-transit` is the long-window stage where a heavy model can earn its cost.
X	Crowding level: Low / Medium / High, or null	`crowding_level` from GVB. **Null on 25% of legs by design.**	Part of the `OPPORTUNITY` composite. High crowding at peak reaches 25.1% of impressions. Its nullability is exactly why Φ exists.
Δt	Delay, minutes	`delay_minutes`, actual minus scheduled. 5 s.	The other half of `OPPORTUNITY`: delay lengthens `W` and raises attention at once. Above five minutes on 12.6% of peak impressions.
K	Local cache state: fresh / stale / empty / evicted	`local_cache_state`. 5 s.	A gate on the rich-history tiers, which cannot compute their features from an empty cache. Non-fresh on **23%** of impressions; low-tier devices are evicted more often, so this correlates with `D`.
H	History depth: prior impressions and tenure	`offers_seen_before`, `tenure_days_at_impression`. Per call.	Routes between the sparse-signal and rich-history models. At the 25th percentile a commuter has seen **one or fewer** prior offers, so this route is well travelled.

Sym	Metric	Source	Decision it justifies
σ	Screen, foreground and motion state	`screen_state`, `app_foreground_state`, `motion_state`. 5 s.	Gates whether to spend anything at all. Locked or backgrounded — and the app is backgrounded on **51.6%** of impressions — drops every task to its cheapest tier or suspends it, because nothing will be seen.
ρ	Location precision: GPS-confirmed / Inferred	`ambient_location_precision` . 5 s.	A confidence qualifier, not a gate. Inferred location (32.4% of impressions) discounts the score of any model whose leading features are geographic, such as `detour_minutes`.

Table 3.3 — Group C. Every row maps to a column that already exists in the LearnPerk Data Dictionary; nothing here needs new instrumentation on the data side, only new reading.

The metric that does not exist in any of the three papers

W, the remaining window. EdgeMLBalancer optimises a stream with no end. HarmonE forecasts a sensor that never stops. CALM serves a queue that refills. LearnPerk has a commuter who alights in four minutes, and after that every millisecond of inference was wasted regardless of how accurate it was. Making W a first-class term is the single largest adaptation this design makes to the published work, and it is what turns an abstract efficiency argument into a concrete one: a model is efficient here if it produces an answer *the commuter is still present to receive.*

3.4 Sampling, aggregation and smoothing

Raw metrics are not fed to the planner. Each is aggregated over the rolling window of n = 12 ticks, in the manner HarmonE prescribes, and each is normalised to [0,1] against a per-device-class reference so that a CPU percentage and a latency in milliseconds can appear in the same weighted sum without one silently dominating.

```
window mean : x_avg = (1/n) * SUM( x_i , i = 1 .. n )
normalise : x_norm = clip( (x - x_min) / (x_max - x_min), 0, 1 )
smooth : EMA_i = beta * x_i + (1 - beta) * EMA_(i-1) , beta = 0.30
```

Three details are worth stating because they are easy to get wrong. First, the normalisation bounds are **per device class**: 40% CPU means something different on a low-tier phone than on a high-tier one, and a shared bound would make the loop behave differently on 30% of the fleet for no principled reason. Second, latency is aggregated as a **p95 reservoir, not a mean**, because the responsiveness boundary is about the tail. Third, gates are evaluated on the **instantaneous** reading rather than the smoothed one: smoothing a connectivity loss over 30 seconds means serving 30 seconds of failures, which is exactly the failure mode EMA smoothing is supposed to prevent elsewhere.

4 The switching decision

Selection happens in four ordered steps. **Gate**, then **score**, then **explore or exploit**, then **commit or hold**. The order is load-bearing: scoring an infeasible model wastes work and, worse, invites a weighted sum to trade away a hard constraint. Nothing downstream can resurrect a candidate the gate rejected.

4.1 Feasibility gate

For a task with spectrum $M = \{m_1 \dots m_n\}$, the gate yields a feasible subset. A model is feasible only if every one of the following holds.

#	Gate condition	Metric	Why it is a gate, not a penalty
G1	Memory: loading it keeps resident model memory under the mode ceiling	**R**	Exceeding it does not degrade the app, it ends the process. There is no accuracy gain worth that trade.
G2	Thermal: the model's tier is permitted at the current thermal status	**T**	Under throttling, every latency figure in the registry is wrong, so the score is no longer measuring what it claims to measure.
G3	Connectivity: if the model needs the network, the network is present	**N**	Binary and immediate. The 22.9% of metro impressions that are Offline make this the most frequently fired gate in the system.
G4	Coverage: available features meet the model's declared minimum set	**Φ**	A gradient-boosted ranker fed nulls for its top splits does not return a worse answer, it returns an arbitrary one, and it returns it confidently.
G5	Window: registry p95 latency for this device class fits the stage budget	**W, L, D**	An answer that arrives after the commuter has alighted has no value at any accuracy.
G6	Delegate: any required NNAPI, GPU or NPU delegate initialised	**Γ**	Without the delegate the model is not slow, it is unavailable.
G7	Health: not currently blacklisted for repeated serve failures	**Ω**	Two failures in one window is evidence about this device, now, which outranks any registry average.
G8	Consent: the model's feature set is permitted by the user's consent state	**Consent**	A withdrawn commuter has no behavioural rows at all, so every personalised tier is not merely discouraged but unusable.

*Table 4.1 — The eight gates. If the feasible set is empty, the task falls to its designated **floor model** — tier 0, offline, no learned features — which is required by registry contract to pass every gate unconditionally.*

4.2 Utility, cost and the composite score

EdgeMLBalancer scores a model as $S = \min(U, U_{avg}) \cdot (1 - C_{avg}/C)$ and takes the argmin. That form does real work — it favours low CPU while rewarding a model whose current confidence beats its own history — but it has one cost channel, one quality channel, and a multiplicative coupling that makes the sign of the score depend on whether confidence is rising. LearnPerk has four cost channels, two immediate quality channels and a context term, so we separate utility from cost additively and take the argmax.

```
[ utility ] Q_m = w_c*Cn_m + w_g*Mn_m + w_a*An_m - w_p*(1 - Phi_m)
    [ cost ] K_m = w_u*Un_m + w_e*En_m + w_l*L95n_m + w_r*Rn_m
[ switch cost ] kappa_m = switch_energy( m_active -> m ) / max( W, W_min )
 [ score ] S_m = alpha*Q_m - (1 - alpha)*K_m - kappa_m + lambda*sim(q, m)
```

The four terms, in order. α is the quality/cost balance and is set entirely by the active operating mode (§4.5) — it is the one knob the decision map controls directly, and it is how a single mechanism produces both switching down under pressure and switching up under opportunity. κ is the switch cost of §2.3, charged only to challengers and divided by the remaining window, which is what makes a heavy model correctly unattractive late in a leg. **λ·sim(q, m)** is CALM's semantic routing term: the cosine similarity between the user's query and the model's task description. It applies only to Task E, where a query has meaning; λ = 0 for Tasks A through D. The incumbent is scored with κ = 0, which is correct rather than a thumb on the scale: staying put genuinely costs nothing.

Scores are then smoothed per model with the EMA of §3.4 before comparison, so the planner compares $\mathrm{EMA}(S_m)$, never a single tick's value.

Weight	Term	Default	Note
w_c	Top-1 calibrated confidence	0.45	The main quality signal, as in EdgeMLBalancer.
w_g	Ranking margin	0.20	Catches confident-but-undiscriminating models.
w_a	Realised accept-rate prior	0.25	Slow-moving; carries the delayed truth without letting it react.
w_p	Coverage shortfall penalty	0.10	Above the gate minimum, a shortfall still discounts.
w_u	CPU	0.30	EdgeMLBalancer's sole cost channel; one of four here.
w_e	Energy per call	0.35	Weighted highest of the cost terms, because the environmental boundary is the one the decision map treats as hardest.
w_l	p95 latency	0.25	Rises sharply as W shrinks — see the stage budgets in Table 4.4.
w_r	Resident memory	0.10	Mostly handled by gate G1; this only breaks ties.
λ	Semantic similarity	0.30 (Task E only)	CALM reports k in 0.1–0.3 across its routing variants; we start at the top of that band because our spectrum is smaller and the domains are further apart.

Table 4.2 — Initial weights. Each group is normalised to sum to 1 within itself, so α alone controls the balance between them. These are starting points for the ablation in §9, not tuned values.

4.3 Dynamic boundaries

A fixed energy ceiling is either too loose to protect the battery or too tight to allow a justified spike. HarmonE's answer, which we adopt unchanged, is a boundary that moves in response to sustained deviation but tolerates transient ones:

```
theta_(i+1) = theta_i + gamma * ( E_ref - E_avg_i ) , gamma = 0.20
```

When measured energy runs above the reference level, the deviation is negative and the ceiling tightens, making heavy models progressively harder to justify. When it runs below, the ceiling loosens and headroom is returned. The commuter-visible consequence is the one we want: a single expensive moment — a long delay, a genuinely valuable offer — is absorbed, while a pattern of expensive moments is progressively squeezed. E_{avg}_ref is set per device class from a two-week baseline, and θ is clamped to a floor so the boundary cannot tighten to the point where only the floor model is ever feasible.

The same mechanism, with its own γ, governs the **latency** boundary. Drift uses a fixed threshold instead (Ψ > 0.25), because a drifting feature distribution is a statement about the world that should not be negotiable against how much battery we have left.

4.4 Exploration, hysteresis and rollback

Pure exploitation of a score table has a known failure: a model that starts poorly is never selected, so its estimate never improves, so it is never selected. EdgeMLBalancer's measured consequence is stark — its naive baseline gave one model 74.3% of all frames and its round-robin baseline gave one model 84.4%, while ε-greedy spread selection across all four and improved fairness by 43.6% and 41.5% respectively. We take the same mechanism for the same reason, then constrain it in two ways the camera case did not need.

1. **Exploration is gated, never unconstrained.** The random draw is over the *feasible* set only. EdgeMLBalancer explores over all four models because all four are always runnable; for us, exploring into a cloud-assisted model in a tunnel is not exploration, it is a guaranteed serve failure.
2. **Exploration is suppressed in Survival mode and when W is small.** The point of exploring is to improve a long-run estimate. At 12% battery, or with 90 seconds of journey left, there is no long run to improve and the cost lands entirely on this commuter.
3. **ε anneals from 0.10 to 0.02** once a model reaches 500 logged calls on this device, because the fairness argument weakens as the estimates firm up.

Hysteresis then stops the loop thrashing. A challenger replaces the incumbent only if all three hold: the incumbent has held the binding for at least the dwell time τ; the challenger's smoothed score exceeds the incumbent's by more than the margin δ; and the challenger passed every gate on this tick. A gate failure bypasses all three conditions, because a gate failure means the incumbent is no longer serveable at all.

Finally, every switch is reversible. The executor warms the target with one synthetic call before swapping the binding. If that call throws, or exceeds the stage latency budget, the switch is abandoned, the incumbent keeps the binding, the target is blacklisted for 90 seconds, and the attempt is logged with `rollback_flag = 1`. A rolled-back switch is a first-class logged outcome, not an error, and its rate is one of the evaluation metrics in §9.4.

4.5 Operating modes

Modes are the decision map made operational. Each fixes α, a ceiling on which tiers are permitted, and whether exploration runs. The analyser selects the mode; the planner works within it. Modes are ordered by pressure, except Opportunity, which is the one mode that moves *up*.

Mode	α	Entry condition	Effect on the spectrum
M1 Full	0.75	B > 60%, T ≤ 1, N = Online, W > 300 s, device High-tier	All tiers permitted. Exploration on. The configuration we compare everything else against.
M2 Balanced	0.55	The default. Anything not matching another mode.	Tiers 0–3 permitted; tier 4 only if the score clears the margin decisively. Exploration on.
M3 Frugal	0.35	B below the user's personal threshold, or T = 2, or ΔB above the dynamic boundary θ	Tiers 0–2 only. Exploration on but restricted to tiers 0–2. Content prefetch suspended.
M4 Survival	0.15	B < 12%, or T ≥ 3, or memory pressure signalled by the platform	Tiers 0–1 only. **Exploration off.** Language layer unloaded entirely. Cached ranking served where possible.
M5 Offline	—	N = Offline. Overlays whichever mode is otherwise active.	Gate G3 removes every network-touching model. The on-device spectrum continues to operate normally within whatever α applies.
M6 Opportunity	0.85	Δt > 5 min **and** W > 600 s **and** B > 50% **and** screen Active. Capped at 5 minutes, then reverts.	Tier ceiling lifted, κ discounted by the enlarged W. This is the mode that **switches up**, and it exists because a delayed passenger with a charged phone is the highest-value moment LearnPerk sees.

Table 4.3 — The six operating modes. M6 is the mode most easily forgotten in a resource-management design, and is where the commercial upside sits.

4.6 The decision table

Collapsing all of the above: each typed uncertainty maps to a tactic and an expected direction of movement. This table is the operational summary of the whole section, and the five use cases in §7 are five traversals of it.

Uncertainty	Raised when	Tactic	Direction
`CONNECTIVITY_LOSS`	N transitions to Offline or Degraded	Gate G3 fires; fall to the best offline-capable model; serve content from the cached pool	**Down**, immediately, no dwell
`RESOURCE_PRESSURE`	U, ΔB or T crosses a boundary, or B crosses the per-user threshold	Enter Frugal or Survival; re-score under the lower α	**Down**, one tier per tick
`WINDOW_CLOSING`	W falls below the stage budget for the active model's p95	Tighten the latency budget; gate G5 removes slow candidates; suppress content entirely below 60 s	**Down**, latency-first
`COVERAGE_LOSS`	Φ drops below the active model's declared minimum	Switch to the model whose minimum feature set is satisfied — usually the sparse-signal sibling, not simply a lighter one	**Sideways**
`QUALITY_DEGRADATION`	EMA(C) or margin M falls below the boundary while resources are ample	Switch to the highest-scoring feasible alternative, excluding the incumbent, exactly as HarmonE's strategy evaluator does	**Up**, if affordable
`DRIFT`	Ψ > 0.25 on a top-10 feature	Switch to the regime-specific model with the closest reference distribution; flag the cloud. **No on-device retraining.**	**Sideways**
`OPPORTUNITY`	The M6 composite condition holds	Raise α to 0.85, lift the tier ceiling, discount κ against the enlarged window	**Up**
`SERVE_FAILURE`	Ω ≥ 2 within one window	Blacklist for 90 s, forced switch, log the rollback	**Down**, immediately, no dwell

Table 4.4 — The decision table. Note that three of eight tactics do not move down the spectrum. A switching engine that only ever switches down is a throttle, not an adaptation loop.

5 The model registry — what models, named

A switching engine is only as good as the spectrum it switches across. If the models are too similar there is nothing to gain by moving between them; if they are too far apart the loop oscillates between unusable and extravagant. EdgeMLBalancer's four detectors — SSD MobileNet V1, EfficientDet Lite0, Lite1 and Lite2 — are a good model for how to space a spectrum: same task, same input, four distinct points on one accuracy-against-cost curve.

This section proposes the LearnPerk equivalent. Every model is named, sized and placed on a tier. Sizes and latencies are **design targets** drawn from published figures for these architectures at these quantisations, not measurements of our own artefacts; the first job after this document is approved is to build the registry and replace every number in it with a measured one.

> **How to read the tiers**
>
> **Tier 0** is not a model — it is a deterministic rule or a lookup table, guaranteed to run anywhere, under any gate, in under a millisecond. Every task must have one, because §4.1 requires a floor that always passes. **Tier 1** is a linear or shallow model. **Tier 2** is a small quantised gradient-boosted model. **Tier 3** is a full gradient-boosted model or a small neural network. **Tier 4** is a neural model that needs a delegate, or a language model. Tier numbers are comparable across tasks, which is what lets an operating mode set one ceiling for the whole system.

5.1 Five task families

Task	What it decides	Called	Spectrum	Floor
A Journey state	Which situation the commuter is in, which is what raises `OPPORTUNITY` and sets the stage	Every 5 s tick	3 models	A0
B Offer ranking	Which offer, at which position, from the eligible merchant set	Per offer surface, a few times per leg	5 models	B0
C Content ranking	Which article or video, in which order	Per feed refresh	4 models	C0
D Incentive policy	How much discount is needed — not which offer, but at what depth	Per offer decision	3 models	D0
E Language layer	Understanding GVB bulletins and free text; answering mid-journey questions	On bulletin arrival, or on a user question	5 models	E0

Table 5.1 — The five task families. Each switches independently, under one shared operating mode and one shared memory budget.

5.2 Task A: journey-state classifier

The cheapest and most consequential model in the system. It runs every tick and its output drives everything else — the stage, the latency budget, and whether the `OPPORTUNITY` condition holds. Classes are: *normal*, *disrupted-delay*, *crowded*, *interchange-imminent*, *leg-ending*, *resource-critical*.

ID	Model	Tier	Size	p95 latency low / mid / high	Needs	When it wins
A0	Threshold rule table — the direct descendant of EdgeMLBalancer's naive baseline, which switches on fixed CPU cut-offs at 20/25/30%	0	~2 KB	<1 / <1 / <1 ms	Nothing. Offline, no features beyond the live feed.	Always feasible; the floor. Also correct on its own terms when delay and crowding are both present and unambiguous.
A1	LightGBM, 30 trees, depth 4, int8	2	~40 KB	3 / 2 / 1 ms	12 context features, no history	The default. Handles the ambiguous middle that a threshold table gets wrong — moderate delay with medium crowding on a short leg.
A2	LightGBM, 200 trees, depth 6, plus line-level and time-of-day interactions	3	~250 KB	9 / 6 / 4 ms	28 features including route history	Peak hours on a known line, where the commuter's own pattern predicts the disruption better than the live feed does.

Table 5.2 — Task A spectrum. Even the heaviest costs under 10 ms, so A switches on accuracy and coverage rather than on resource pressure.

5.3 Task B: offer ranking

The commercially important task, and the one the dataset is built around: `accepted` is the stated target variable and `truth_p_accept` is the stated ceiling. The spectrum spans a lookup table to a two-tower network, and it is deliberately wide because the conditions span a tunnel on a low-tier phone to a delayed tram on a high-tier one.

ID	Model	Tier	Size	p95 latency low / mid / high	Needs	When it wins
B0	**Segment-prior table.** Precomputed acceptance rates by (age cell × merchant category × peak flag × discount band), shipped with the app and refreshed nightly	0	~60 KB	<1 / <1 / <1 ms	Nothing about this commuter. Offline-safe.	The floor, and the honest answer at true cold start. A commuter on their first impression has no history to personalise on, and 17% of impressions have a null `accept_rate_before`.
B1	**L2 logistic regression**, 48 features, int8 weights	1	~25 KB	2 / 1 / <1 ms	Context and offer features; tolerates missing history	Latency-critical moments. When the window is under 60 s, the right question is not which model is most accurate but which one answers in time.
B2	**LightGBM, 100 trees, depth 4, int8** — the workhorse	2	~180 KB	7 / 4 / 3 ms	64 features; minimum coverage 0.7; tolerates a null `crowding_level`	The default in Balanced mode, and the offline default. Captures the discount-by-price-sensitivity interaction that B1 cannot.
B3	**LightGBM, 500 trees, depth 8**, with explicit detour, goal-gradient and repeat-merchant interactions	3	~1.4 MB	26 / 14 / 9 ms	96 features; minimum coverage 0.9; needs a fresh cache	Full and Opportunity modes. The dictionary is explicit that detour is the main channel through which home zone changes what people buy, and that it enters as a *threshold*, not a slope — a shape that needs depth to represent.
B4	**Two-tower network**, 32-d commuter and merchant embeddings, TFLite int8	4	~3.2 MB	n/a / 48 / 28 ms	NNAPI or GPU delegate; fresh cache; tenure > 30 days	Opportunity mode on a high-tier device, where the embedding can generalise to a merchant this commuter has never seen. Infeasible on low-tier by registry contract.

Table 5.3 — Task B spectrum. Note that B0 and B2 are the only two guaranteed to work offline, which is why they carry the metro case.

5.4 Task C: content ranking

Content has the capability hole identified in Table 1.1: zero views occur offline today. The spectrum is therefore designed around a **locally cached item pool** rather than a live catalogue, so that C0 and C1 can serve a tunnel.

ID	Model	Tier	Size	p95 latency low / mid / high	Needs	When it wins
C0	**Recency-and-category shortlist** over the cached pool, ordered by the commuter's declared onboarding interests	0	~4 KB	<1 ms all tiers	The cached pool only. Offline-safe.	Offline, Survival mode, or an empty cache. Closes the hole in row 2 of Table 1.1.
C1	**Taste-vector cosine** — dot product of the ten-weight running taste vector against each cached item's category vector, length-adjusted against the remaining window	1	~8 KB	1 / <1 / <1 ms	The taste vector and item categories. Offline-safe.	The offline default. Directly addresses the recurring free-text complaint quoted in the v3 document — *this article was too long for a 12-minute tram ride* — because `length_units` is scored against W.
C2	**LightGBM dwell-and-completion ranker**, 150 trees, depth 5, int8	2	~220 KB	8 / 5 / 3 ms	40 features including dwell history; coverage 0.7	The online default. Completion is only 21.5% and bounce 10.7%, so predicting *which items survive the first two seconds* is worth real model capacity.

ID	Model	Tier	Size	p95 latency low / mid / high	Needs	When it wins
C3	**MiniLM-L6 sentence embeddings** (int8) over item text, with approximate nearest-neighbour search against a commuter interest vector	4	~23 MB	n/a / 62 / 35 ms	Delegate; embeddings precomputed on Wi-Fi	Full mode on a long leg. Generalises to items with no engagement history at all, which the category-level models cannot.

Table 5.4 — Task C spectrum, built around a cached item pool so the bottom two tiers survive a tunnel.

5.5 Task D: incentive policy

Task D answers the question the v3 document frames precisely: not *what* to offer but *how much incentive* is needed to change behaviour. John declining 5% and saying so in free text is a labelled counterfactual. The arms are the six discount depths the dictionary records: 5, 10, 15, 20, 25 and 30 percent.

ID	Model	Tier	Size	p95 latency low / mid / high	Needs	When it wins
D0	**Merchant default ladder** — take the depth the merchant configured, with no adjustment	0	n/a	0 ms	Nothing. Offline-safe.	The floor, and correct whenever we have no basis to deviate. Also the control arm for measuring whether D1 and D2 pay for themselves.
D1	**Contextual bandit (LinUCB)** over the six discount arms, with context = {tier progress, price-sensitivity estimate, detour, expiry window, peak flag}	2	~50 KB	3 / 2 / 1 ms	Per-user posterior; updates on every resolved outcome	The default once a commuter has roughly 20 resolved impressions. Cheap, well understood, and gives an honest uncertainty estimate, which matters because it is spending margin.
D2	**Offline-trained policy network** (conservative Q-learning on logged data, 3-layer MLP, TFLite int8)	3	~900 KB	18 / 10 / 6 ms	Full feature set; a policy trained centrally and shipped; fresh cache	Full and Opportunity modes for rich-history commuters. Represents the sequential structure a bandit cannot — that a shallow discount now may be better if it preserves the goal-gradient pull toward the next 500-point reward.

Table 5.5 — Task D spectrum. Every arm of every tier is logged with its propensity, so the whole task remains off-policy evaluable (§9.3).

5.6 Task E: the natural-language layer

This is where CALM applies almost directly. The v3 document sets out the material: GVB bulletins arriving as unstructured Dutch and English text, free-text onboarding and preference statements, merchant offer copy, and — the clearest case — commuters asking the assistant questions mid-journey. *Why is my tram delayed? Is it worth waiting or should I walk? Will I make my connection at Zuid?*

CALM's finding is that a fleet of small specialised models beats one large general one on latency, energy and domain confidence. Its constraint is memory, solved with an LRU cache. On a phone the same constraint is far tighter: **our cache size is one**. At most one language model is resident, and evicting it to load another is a genuine cost the score must pay for — which is exactly what κ in §4.2 does.

ID	Model	Tier	Size (int4/int8)	p95 latency mid / high	Needs	When it wins
E0	**Bilingual phrase and pattern table** over GVB bulletin vocabulary — *signaalstoring*, *vertraging*, *werkzaamheden*, *pendelbussen* — mapped to structured disruption facts	0	~120 KB	<2 ms	Nothing. Offline-safe.	The floor, and genuinely sufficient for most bulletins, which are formulaic. Extracts line, cause and expected delay without a neural model.
E1	**MiniLM-L6 int8 embeddings** for intent classification and CALM-style cosine routing across a fixed intent set	2	~23 MB	34 / 20 ms	Loaded once; no delegate required	Classifying a free-text preference statement or routing a question. Also computes the `sim(q, m)` term that the rest of Task E routes on.

ID	Model	Tier	Size (int4/int8)	p95 latency mid / high	Needs	When it wins
E2	**Gemma 3 270M Instruct**, int4	4	~200 MB	~340 / 190 ms per short answer	Warm residency; 320 MB peak RSS	Short generated answers offline. Handles the transit Q&A pattern well because the answers are templated facts wrapped in a sentence.
E3	**Qwen2.5-0.5B-Instruct** or **Llama-3.2-1B-Instruct**, int4	4	~400 MB / ~750 MB	n/a / 520 ms	High-tier only; delegate; charging or B > 50%	Full mode on a high-tier device, for multi-clause reasoning — *will I make my connection* requires combining delay, remaining window and timetable.
E4	**Cloud assist** — Claude Haiku 4.5 for interactive turns, Claude Sonnet 5 for the nightly batch that re-derives taste vectors and consent-safe summaries	cloud	n/a	400–900 ms	N = Online; consent; user-initiated only	A genuinely open-ended question on a good connection. Gate G3 removes this tier on 22.9% of metro impressions, which is the whole reason E0 to E3 exist.

Table 5.6 — Task E spectrum, ordered as CALM orders a fleet: by capability, with residency as the binding constraint rather than raw speed.

Where we depart from CALM, and why

CALM routes across a fleet held simultaneously on one 48 GB GPU and reports that shrinking the cache to one model cut peak memory from 32.5 GB to 7.9 GB while *still* beating single-LLM baselines on latency. That result is the licence for our design: a cache of one is not a compromised version of CALM, it is the configuration CALM showed still works. What changes on a phone is that eviction is no longer cheap, so we do two things CALM does not. We charge the reload cost to the challenger through κ, and we set Task E's dwell time to 120 s rather than 20 s, so the language layer switches roughly once per leg rather than once per tick.

5.7 Residency, memory budget and caching

Tasks A to D are small enough to hold all of their tiers resident: the entire A, B, C and D registry is about 6 MB, which fits any device class. Task E dominates the budget by two orders of magnitude, so residency policy is really Task E policy.

Device class	Model memory ceiling	Always resident	Cache	Language tier permitted
Low-tier (30% of fleet)	160 MB	All of A, B, C, D (~6 MB)	E0 only; E1 loaded on demand and evicted after use	E0, E1
Mid-tier (45%)	280 MB	All of A, B, C, D, plus E1	One slot for E2, LRU, evicted under Frugal mode	E0, E1, E2
High-tier (25%)	900 MB	All of A, B, C, D, plus E1	One slot for E2 or E3, LRU	E0, E1, E2, E3

Table 5.7 — Residency by device class. The single-slot language cache is the direct analogue of CALM's cache_size = 1 configuration.

Three residency rules follow, and all three exist because a cold start is visible to the commuter. First, **the language slot is preloaded at leg start**, not on first question, using the journey-state classifier's prediction of whether a question is likely — a delayed leg makes a question far more likely. Second, **eviction happens at leg boundaries** wherever possible, because that is when nobody is waiting. Third, **Survival mode unloads the slot entirely** and does not reload it until charge recovers, since at 12% battery the correct answer to an open-ended question is a templated one from E0.

6 What data each model needs

The switching engine cannot choose a model it cannot feed. This section states, per task and per tier, which fields are required, which are optional, and what the declared minimum coverage is — the number gate G4 compares against. Every field named here already exists in the LearnPerk Data Dictionary; §8 covers what has to be *added*.

6.1 Feature sets by task

Task / tier	Required fields	Optional (improves but not required)	Min. coverage
A0	`delay_minutes, duration_minutes, motion_state, battery_pct, connectivity_state`	none	1.00, but all five are always present
A1	A0 set plus `crowding_level, is_peak, hour, board_stop_type, alight_stop_type, mode, screen_state, n_legs`	—	0.75 — tolerates the 25% null crowding
A2	A1 set plus `line_id, origin_zone, destination_zone, day_of_week` and the commuter's own history on this line	`ambient_location_precision`	0.90
B0	`age_cell, merchant_category, is_peak, discount_pct`	none	1.00 — all four are known at impression time
B1	B0 set plus `detour_minutes, expiry_window_hours, position, tier_at_impression, tier_progress, mode, battery_pct, connectivity_state`	`crowding_level, delay_minutes`	0.60
B2	B1 set plus `merchant_seen_before, offers_seen_before, occupation, household_type, traveler_archetype, home_zone, work_zone, merchant_zone, mcc, merchant_partner_type, purpose`	`accept_rate_before` (17% null), `crowding_level` (25% null)	0.70
B3	B2 set plus `offers_accepted_before, accept_rate_before, repeat_purchase_flag, points_balance_after, basket_value_eur` history, and the derived price-sensitivity and detour-tolerance estimates	—	0.90, and `local_cache_state = fresh`
B4	B3 set plus a trained commuter embedding, which needs at least 30 days of tenure and 20 resolved impressions	—	0.95, fresh cache, delegate present
C0	`display_category` of cached items, declared onboarding interests, `shown_at`	none	1.00
C1	C0 set plus the running taste vector and `length_units, format, duration_minutes`	`iab_tier1`	0.80
C2	C1 set plus `dwell_seconds, completion_state, bounced, reaction, position, hour, is_weekend` history	`has_reaction` (96% null — rare by design)	0.70
C3	C2 set plus precomputed item text embeddings	—	0.85, delegate, embeddings cached
D0	`discount_pct` as configured by the merchant	none	1.00
D1	`tier_progress, detour_minutes, expiry_window_hours, is_peak, merchant_category` plus the per-user arm posterior	—	0.80 and at least 20 resolved impressions
D2	D1 set plus full purchase and points history and a centrally trained policy	—	0.90, fresh cache
E0	The bulletin text, plus `line_id` for relevance filtering	`language_preference`	1.00

Task / tier	Required fields	Optional (improves but not required)	Min. coverage
E1	Query or bulletin text, plus the model description strings for routing	—	1.00
E2 / E3	Query text, the structured journey facts E0 extracted, and the current offer and points state for grounding	Conversation history within the leg	1.00 plus residency
E4	As E2, plus a consent-checked and minimised context payload	—	Online, consent, user-initiated

Table 6.1 — Feature sets and minimum coverage by tier. The minimum-coverage column is not decoration: it is the literal parameter gate G4 evaluates, and it is why a null `crowding_level` *routes to B2 rather than breaking B3.*

6.2 Coverage, nulls and graceful degradation

The Data Dictionary makes a point worth repeating here, because the whole coverage mechanism rests on it: *in this dataset a null is always meaningful — either consent said no, the event never happened, or a merchant did not disclose something. Filling nulls with zero collapses three different facts into one.* A switching engine that imputes zeros to keep a heavy model feasible is doing exactly the thing the dictionary warns against, and it will do it silently and confidently.

So coverage is handled by routing, not by imputation. Three rules:

1. **A null below the minimum routes sideways, not down.** When `crowding_level` is absent, the right move is not B3 to B2-because-it-is-cheaper; it is B3 to B2 *because B2 was trained without depending on that split*. Tier is a cost ordering; coverage is a capability question, and they are not the same axis.
2. **Above the minimum, a shortfall is a discount.** A model at 0.75 coverage against a 0.70 minimum is feasible but penalised by `w_p · (1 − Φ)`, which is often enough to let a fully covered lighter model win outright. That is the correct outcome.
3. **Structural nulls are declared, not discovered.** The registry records, per model, which fields are expected to be null and how often — 25% for crowding, 17% for `accept_rate_before`, 84% for the whole purchase block on any single impression. A model surprised by a null it was told to expect is a model that was trained wrong.

One consequence deserves stating plainly. The commuters most likely to hit low coverage — new accounts, low-tier devices with evicted caches, offline metro legs — are exactly the commuters we most want to retain. The sparse-signal path is not a degraded service for unimportant users; it is the first experience most users have. B0 and C0 deserve the same design attention as B3.

7 Five use cases

Each case below is one traversal of the decision table in §4.6. The structure is the same throughout: who the commuter is, what they do in LearnPerk, what happens at a specific moment, which metrics read what, which model each task moves to and why, what the efficiency argument is in numbers, and how we would test that the engine actually did it. The personas are constructed, but every profile attribute and every metric value is drawn from a real range in the dataset.

The five are chosen to exercise different arms of the decision table deliberately. Two switch down, one switches up, one switches sideways, and one is mostly about a switch correctly *refused*. A set of five cases that all switch down would demonstrate a throttle, not an adaptation loop.

7.1 UC-1 Lisa — the tunnel

The switching arm: `CONNECTIVITY_LOSS`**. Direction: down and sideways, immediately, with no dwell-time grace.**

User profile

- **Lisa Bakker**, 34. `age_cell` A4, Established professional, couple no kids.
- Home zone **West**, work zone **Zuid**. Habitual commuter, 640 days tenure.
- Device: **Mid-tier**. Consent opted-in, no accessibility flag.
- Declared interests at onboarding, in her own words: *“I travel to Zuid most days for work, I like trying new coffee places and I’m trying to eat healthier.”*

LearnPerk engagement

- 27 offers seen, 6 accepted — an accept rate of 22%, comfortably above the 15.5% dataset mean.
- Top content category: Local News & Current Affairs. Has asked for GVB service updates specifically.
- Has said she wants **variety**: *“still my thing, but I’d like more variety, not always the same chain.”*
- Tier Gold, `tier_progress` 0.88 — close enough to the next reward that the goal-gradient term is strongly active.

The moment. 08:18 on a Tuesday, metro M52 from Zuid towards Centraal. The leg is scheduled at 14 minutes and she boarded at the origin, so the window is long. Three minutes in, the train enters the tunnel under the IJ. Connectivity goes from `Online` to `Offline` between one tick and the next. This is not a rare event: it is 22.9% of metro-leg impressions in our own data.

Metric	Reading at the moment	Boundary	Verdict
N connectivity	`Online` → `Offline` at t = 185 s	Gate G3, instantaneous, unsmoothed	**Gate fails** for every network-touching model
K cache state	`fresh` → `stale` 40 s later, once the catalogue refresh times out	B3 requires `fresh`	**Gate fails** for B3
Φ coverage	B3 drops from 0.94 to 0.61 as live merchant and crowding fields go stale	B3 minimum 0.90; B2 minimum 0.70	B3 infeasible, B2 feasible at 0.78
W remaining window	~660 s	—	Ample. Latency is not the constraint here.
B battery	56%	Her personal threshold, learned from past disengagement: 31%	No resource pressure. Mode stays **Balanced**, overlaid by **M5 Offline**.
U CPU	11% smoothed	Balanced ceiling 35%	Comfortable

Table 7.1 — UC-1 readings. Note that only two metrics moved, and both are gates. No score comparison was needed to make the first three switches — the gate decided them.

Task	Switch	Trigger	Why this model, on these metrics
B offers	B3 → **B2**	G3 then G4	B2 is the heaviest model in the spectrum that is both offline-capable and satisfied at 0.78 coverage. This is a sideways move for an epistemic reason and a downward move for a resource reason at the same time, and B2 is the answer to both. The goal-gradient feature that matters most for Lisa — `tier_progress` at 0.88 — is local and survives the outage, so the ranking stays personal.

Task	Switch	Trigger	Why this model, on these metrics
C content	C2 → **C1**	G3	C2 needs the live catalogue. C1 runs a cosine over the taste vector against the *cached* pool and adjusts for remaining window. This is the switch that closes the hole in Table 1.1 row 2: today Lisa would see **nothing** for eleven minutes, because zero content views in the dataset occur offline.
E language	E4 → **E0 + E1**	G3	Cloud assist is gone. The GVB bulletin she cares about is formulaic Dutch — *signaalstoring, vertraging van circa 10 minuten, lijn 24* — and E0's phrase table extracts line, cause and delay from it without any neural model. E1 stays resident on mid-tier and handles intent routing at ~34 ms.
A journey state	A1 → **A1** (hold)	—	A1 needs nothing the outage removed. Holding is the correct decision and the score produces it without a special case.
D incentive	D1 → **D1** (hold)	—	The LinUCB posterior is local. Nothing about being offline makes it worse.

Table 7.2 — UC-1 switches. Three of five tasks move; two correctly hold.

Why this is efficient

The efficiency argument here is not primarily about energy — it is about **availability**, which is the goal the decision map calls *never go dark*. Without switching, Lisa gets a blank content feed and a dead assistant for eleven minutes of a fourteen-minute leg, and we record nothing, so the failure is invisible. With switching she gets a cached, taste-ranked feed and a structured answer about her delay. The secondary gain is real but smaller: B2 at ~180 KB and 4 ms replaces B3 at ~1.4 MB and 14 ms, roughly a **3.5× cut in per-call inference cost**, and dropping the cloud round trip removes a 400–900 ms wait that would in any case have failed. Switching costs are negligible: B2 and C1 are already resident, so κ is near zero and the gate bypasses dwell time legitimately.

How we test this case

- **Trigger test.** Replay a metro leg from `event_stream.csv` with connectivity forced to Offline at t = 185 s. Assert that gate G3 fires on the *same tick*, not the next, and that dwell time is bypassed.
- **Selection test.** Assert the chosen model is B2, not B1. A down-switch that overshoots to the cheapest available model is the most likely implementation bug, and it costs accuracy for no reason, since there was no resource pressure at all.
- **Availability test.** Assert content items are returned for every tick of the outage. The pass condition is a non-empty feed, measured against a baseline of zero.
- **Recovery test.** On exiting the tunnel, assert that the engine does *not* immediately restore B3: the cache is still stale, and the dwell timer on B2 has not expired. Restoring early is the second most likely bug.

7.2 UC-2 John — the failing battery

The switching arm: `RESOURCE_PRESSURE`**. Direction: down, progressively, one tier per tick.**

User profile

- **John de Vries**, 51. `age_cell` A7, Established professional, solo household.
- Home zone **Noord**, work zone **Centrum**. Habitual commuter, metro every day, 410 days tenure.
- Device: **Low-tier** — the 30% of the fleet whose cache is evicted more often.
- `accessibility_flag = hearing_impairment`. In his own words: *"please make sure alerts are visual, not just audio."*

LearnPerk engagement

- Low-volume, high-signal: 21 offers seen, 3 accepted. Explicitly asked for **minimal offers**.
- Stated preferences: *"quick useful info, not too many offers"*; *"don't send me food offers before 9am"*.
- Declined a 5% discount and said why: *"I'd redeem this if the discount was bigger — 5% isn't enough to change my mind."* That is a labelled counterfactual for Task D.
- Estimated `battery_anxiety` 0.61, putting his personal disengagement threshold at **24%** within the documented 20–40% band.

The moment. 18:40, the evening leg home on M52. He has been navigating on a low-tier phone all afternoon. Charge is 18% and falling at 4.6% per hour, the thermal status has reached `LIGHT`, and the leg has 14 minutes to run. This combination — low-tier and under 30% charge — is 5.8% of impressions in our data, which is roughly one commuting session in seventeen.

Metric	Reading at the moment	Boundary	Verdict
B battery	18%	His personal threshold: 24%. Global Survival floor: 12%	**Below personal threshold.** Mode Balanced → **Frugal**, α 0.55 → 0.35
ΔB drain rate	4.6%/hr, near the top of the measured 1.06–5.02 band	Dynamic boundary θ, currently tightened to 3.9%/hr after two sustained overspend windows	**Boundary exceeded.** θ tightens again on the next update
T thermal	1 (`LIGHT`)	Tier 3+ blocked at 2	Not yet gating, but one step away
D device class	Low-tier	Memory ceiling 160 MB; no language tier above E1	E2 and E3 were never feasible on this device
U CPU	29% smoothed, against a 22% baseline for this device	Frugal ceiling 25%	**Exceeded**
W remaining window	~840 s	—	Long. Latency is not the constraint; energy is.

Table 7.3 — UC-2 readings. Three separate resource signals agree, which is what distinguishes a genuine regime change from the noise that EMA smoothing exists to absorb.

Task	Switch	Trigger	Why this model, on these metrics
B offers	B2 → **B1**	Frugal mode; U above ceiling	At α = 0.35 the cost term dominates. B1 is ~25 KB against B2's ~180 KB and under 2 ms against 7 ms on this device class. The expected acceptance loss is small for John specifically, because his own free text says depth of discount — a feature B1 carries directly — is what moves him, not the subtle interactions B2 adds.
C content	C2 → **C1**	Frugal mode	C1 is a dot product over a vector he has been building for 410 days, so the ranking stays genuinely personal at almost no cost. Content prefetch is suspended, which is where most of the saving actually comes from.
E language	E1 → **E0**, slot unloaded	Frugal; memory ceiling	E1's ~23 MB is unloaded, freeing the largest single block on a 160 MB ceiling. E0 still parses the GVB bulletin John actually wants, and — because of his accessibility flag — renders it **visually**. The flag changes rendering, never which model runs.

Task	Switch	Trigger	Why this model, on these metrics
D incentive	D1 → **D1** (hold)	—	**The instructive non-switch.** D1 is ~50 KB and about 2 ms. Dropping it to D0 would save an amount of energy indistinguishable from measurement noise and would cost the one thing John has told us matters — getting the discount depth right. The score sees this because D1's cost term is near zero, so even at $\alpha = 0.35$ its utility still wins. A rule-based throttle would have dropped it.
A journey state	A1 → **A0**	U above ceiling	On a familiar daily route the threshold table is very nearly as good as A1, and it costs nothing. This is the one place where EdgeMLBalancer's naive baseline is genuinely the right answer, and the score says so.

Table 7.4 — UC-2 switches. Four tasks move down; D holds. The held task is the point of the case.

Why this is efficient

Summed across the five tasks, resident model memory falls from roughly 24 MB to under 300 KB, which on a 160 MB ceiling removes essentially all of the model-attributable pressure. Per-surface inference cost falls from about 16 ms to about 3 ms on this device class. EdgeMLBalancer measured 2.10 mAh for its adaptive policy against 5.25 mAh for a naive threshold policy over an identical 30-minute workload — a **60% reduction** — and our structure is the same, so a comparable order of saving is the hypothesis §9 is designed to test. The argument that matters for John, though, is behavioural rather than electrical: his personal disengagement threshold is 24%, and he is at 18%. Every mJ LearnPerk does not spend is charge that keeps him in the session at all.

How we test this case

- **Personal-threshold test.** Two commuters at identical 22% charge, with anxiety-derived thresholds of 21% and 34%. Assert that only the second enters Frugal. A global constant would fail this, and it is the failure the dictionary explicitly warns about.
- **Non-switch test.** Assert D1 is still bound after ten ticks in Frugal mode. This is the regression test that catches a blanket throttle disguised as a switching engine.
- **Hysteresis test.** Oscillate charge across the threshold, 23%–25%, for two minutes. Assert at most one mode change, and that the dwell timer and the margin δ are both respected.
- **Accessibility invariance test.** Assert the rendering path for `hearing_impairment` is identical across every tier from E0 to E4. Accessibility must never be a casualty of a down-switch.
- **Energy test.** On-device, two matched 30-minute runs, fixed workload, 30-minute cooldown between them, following the EdgeMLBalancer protocol. Compare mAh.

7.3 UC-3 Sanne — the delayed, crowded train

The switching arm: `OPPORTUNITY`**. Direction: up. This is the case most resource-management designs forget.**

User profile

- **Sanne Visser**, 27. `age_cell` A3, Early-career professional, shared housing.
- Home zone **Oost**, work zone **Zuid**. Habitual commuter, 395 days tenure.
- Device: **High-tier**. Opted-in, no accessibility flag.
- Estimated novelty-seeking 0.71 and detour tolerance ~9 minutes — both well above median, so she is unusually open to an unfamiliar merchant slightly off her route.

LearnPerk engagement

- 24 offers seen, 5 accepted. Mean discount seen 14%, mean detour seen 5.2 minutes.
- Content: high video share, strong appetite (multiplier 1.9), median dwell 41 s.
- Tier Silver, `tier_progress` 0.41.
- Asks the assistant questions when disrupted — her history shows three questions in the last month, all on delayed legs.

The moment. 17:52, peak, Tram 12 towards Centraal. The GVB feed reports a signal problem: `delay_minutes` is 11 and `crowding_level` is `High`. She is standing, screen active, phone at 74%. The delay has stretched the remaining window from 13 minutes to about 22. She is, for the next twenty minutes, a captive and attentive audience on a fast phone with plenty of charge.

Metric	Reading at the moment	Boundary	Verdict
Δ**t** delay	11 minutes	M6 requires > 5 minutes. Only 12.6% of peak impressions clear this	**Condition met**
W remaining window	~1,320 s, against a median leg of 714 s	M6 requires > 600 s	**Condition met**, and generously
B battery	74%	M6 requires > 50%	**Condition met**
σ screen / foreground	Active, foreground	M6 requires Active	**Condition met.** The app is backgrounded on 51.6% of impressions, so this is the minority state that makes spending worthwhile
X crowding	High	Not a gate; a utility signal	In our data, acceptance under High crowding is **16.5%** against **14.8%** under Low — 1.7 pp higher, small but real
T / **U**	Thermal 0, CPU 14%	Full ceiling 45%	No pressure. Tier 4 is available
κ switch charge	Loading B3 costs ~110 mJ; amortised over 1,320 s it is ~0.08 mJ/s	Compared against the utility gain from α = 0.85	**Negligible.** The long window is precisely what makes the upgrade affordable

Table 7.5 — UC-3 readings. Every M6 condition holds simultaneously, which is rare and is meant to be: M6 is capped at five minutes and then reverts.

Task	Switch	Trigger	Why this model, on these metrics
A journey state	A1 → **A2**	M6	With 28 features including her own history on line 12, A2 can distinguish *this delay will clear* from *this delay will grow*, which determines how much window the other tasks can rely on.
B offers	B2 → **B3**	M6	B3's depth-8 trees carry the explicit detour, goal-gradient and repeat-merchant interactions. Sanne has ~9 minutes of detour tolerance and high novelty-seeking, and the dictionary is explicit that detour enters as a **threshold rather than a slope** — the hardest trait to recover, and the shape that most needs depth. This is precisely the commuter and the moment where B3's extra capacity converts into an acceptance that B2 would miss.
B (considered)	B3 → B4 **rejected**	Score, not gate	B4 is feasible: high-tier, delegate present, tenure 395 days. It loses on score. Its ~3.2 MB load and 28 ms p95 buy generalisation to unseen merchants, but Sanne has 24 prior impressions in familiar categories, so the embedding has little to add. The margin δ is not cleared. **Feasible is not the same as selected.**

Task	Switch	Trigger	Why this model, on these metrics
D incentive	D1 → **D2**	M6	At `tier_progress` 0.41 she is mid-ladder, where the sequential question is live: a shallower discount now may be worth more if it preserves the pull toward the 500-point reward. A bandit optimises this impression; D2 optimises the sequence.
E language	E1 → **E2 preloaded**	M6 + A2 prediction	A2 predicts a high probability of a question on a delayed leg, and her history agrees. E2 is loaded into the single language slot **now**, while she is not waiting, so the answer to *"will I make my connection at Zuid?"* is warm when it is asked. This is the preload rule of §5.7 doing its job.

Table 7.6 — UC-3 switches. Four tasks move up; one candidate is feasible but correctly rejected on score.

Why this is efficient

Efficiency is a ratio, not a floor, and this case is the numerator. The inference spend rises roughly **4×** for the next five minutes — and this is the moment when a marginal accepted offer is worth most. The three conditions that make it affordable are all measured rather than assumed: the window is 85% longer than a median leg, so κ amortises to almost nothing; the battery is 74%, so the energy has no behavioural cost; and the screen is active, which is the minority state. A design that only ever switches down would spend exactly the same at 17:52 as at 03:00, and would be leaving the entire upside on the table. M6 is capped at five minutes and reverts automatically, so the exposure is bounded by construction.

How we test this case

- **Composite-trigger test.** Assert M6 fires only when *all four* conditions hold. Negative cases: delay 11 min but battery 40%; delay 11 min but screen locked; delay 3 min with everything else true. None may fire.
- **Cap test.** Assert M6 reverts after 300 s even if every condition still holds, and that it cannot immediately re-enter.
- **Rejection test.** Assert B4 is *scored* and *rejected*, not gated out. The log must show it in the feasible set with a score below B3's. Confusing the two makes the system untunable, because a gate cannot be re-weighted.
- **Value test.** Off-policy, using the `propensity` column: estimate the acceptance lift of B3 over B2 restricted to impressions where the M6 condition holds. If the lift is not positive, M6 is not worth its cost and the mode should be retired.
- **Preload test.** Assert E2 is resident *before* the first question, and measure the saved cold-start latency — the whole justification for preloading.

7.4 UC-4 Daan — the commuter we know nothing about

The switching arm: `COVERAGE_LOSS`**. Direction: sideways. Resources are plentiful; the constraint is evidence.**

User profile

- **Daan Mulder**, 19. `age_cell` A2, Student, shared housing.
- Home zone **Centrum**, university in **Nieuw-West**. Occasional/personal archetype.
- Device: **Mid-tier**. Battery 63%, thermal 0 — **no resource pressure whatsoever**.
- Tenure **9 days**. The dataset's 10th percentile is 26 days, so he is newer than 90% of impressions.

LearnPerk engagement

- `offers_seen_before` = 1. `accept_rate_before` = **null** — undefined, not zero, and null on 17% of all impressions.
- `local_cache_state` = **empty**. No behavioural history has accumulated locally yet.
- 4 content views, no reactions. Reaction rate across the whole dataset is only 3.6%, so this tells us almost nothing either way.
- One free-text statement at onboarding: *"I'm a student, budget offers are more useful to me than premium ones."* Right now this is the single most informative thing we hold about him.

The moment. 14:20 on a Thursday, bus 21 towards Nieuw-West, a 16-minute leg, online throughout. Nothing is under pressure. The phone is fine, the network is fine, the window is long. The heavy models are all *feasible* — and all of them would be wrong, because there is nothing for them to condition on.

Metric	Reading at the moment	Boundary	Verdict
H history depth	`offers_seen_before` = 1, tenure 9 days	B3 requires meaningful history; B4 requires 30 days and 20 impressions	**B4 gated out.** B3 technically loads but conditions on near-empty features
K cache state	`empty`	B3 requires `fresh`	**Gate G4 fails** for B3
Φ coverage	B3 at 0.44; B2 at 0.58; B1 at 0.71; B0 at 1.00	Minimums: B3 0.90, B2 0.70, B1 0.60, B0 1.00	**Only B1 and B0 are feasible**
B, **U**, **T**, **N**	63%, 12%, 0, Online	All comfortable	**No resource pressure at all.** Mode stays Balanced
W remaining window	~900 s	—	Long
D1 posterior depth	1 resolved impression	D1 requires ~20	**D1 not yet usable.** Its uncertainty estimate would be all uncertainty

Table 7.7 — UC-4 readings. Every resource metric is healthy. The constraint is entirely epistemic, which is why the coverage metric Φ has to exist separately from the cost terms.

Task	Switch	Trigger	Why this model, on these metrics
B offers	B2 → **B0**, then B1 at ~impression 8	G4 coverage	B0 is a segment-prior table: acceptance by age cell, merchant category, peak flag and discount band. For a 19-year-old student at 14:20 it returns the population's honest estimate rather than a personalised guess built from one data point. As impressions accumulate, coverage crosses B1's 0.60 minimum and the engine promotes him — a switch driven by *evidence arriving*, which is the reverse of every other case in this section.
C content	C2 → **C0**, then C1 at ~view 12	G4 coverage	C0 orders the cached pool by his declared onboarding interests, because declared interest is genuinely all we have. C1 takes over once the taste vector has enough mass to beat the declared prior — which is exactly the refinement loop the v3 document describes, where the system later asks *"still your thing, or should we show you less of it?"*
D incentive	D1 → **D0**	Insufficient posterior	With one resolved outcome, LinUCB would be exploring at full width and spending real margin to learn almost nothing. The merchant default ladder is the correct, honest floor, and it doubles as the control arm for measuring D1's value later.

Task	Switch	Trigger	Why this model, on these metrics
E language	E0 → **E1** *(a switch up)*	Sparse structured signal	**The counter-intuitive move.** When structured history is empty, the free text is the richest thing we hold. E1 embeds *"budget offers are more useful than premium ones"* and seeds an initial taste and price-sensitivity prior. Spending 23 MB and 34 ms once, on the one signal that exists, is a far better use of the budget than running a heavy ranker over nothing.

Table 7.8 — UC-4 switches. Three tasks move down in tier and one moves up, all from the same trigger. This is why coverage cannot be collapsed into cost.

Why this is efficient

Two arguments, and the second is the important one. The narrow argument: B0 at ~60 KB and under a millisecond replaces B2 at ~180 KB and 4 ms, for an expected accuracy *gain* rather than a loss, because B2 conditioned on empty features is worse than a well-estimated population prior. Spending less and getting more is the rare case, and cold start is where it occurs. The broader argument: the failure this avoids is not slowness, it is a **confident wrong answer**. A gradient-boosted model fed nulls for its top splits does not return a hedged answer; it returns an arbitrary one with a high confidence score attached, and that score then poisons the switching engine's own quality metric. For a nine-day-old account, an irrelevant offer shown confidently is the most expensive thing LearnPerk can do, because the cost is churn rather than milliamps.

How we test this case

- **Null-handling test.** Assert that a null `accept_rate_before` is never imputed as zero anywhere in the feature pipeline. Test the three distinct meanings the dictionary names — consent refused, event never happened, merchant did not disclose — and assert they route differently.
- **Direction test.** Assert the engine moves B down and E *up* from the same trigger on the same tick. A coverage handler that only moves down has conflated coverage with cost.
- **Promotion test.** Feed impressions one at a time and assert the promotion to B1 happens when Φ crosses 0.60, not on a fixed impression count and not on tenure.
- **Calibration test.** Compare B0's predicted acceptance against realised acceptance for all impressions with tenure under 14 days. A prior that is not calibrated is not a prior.
- **Counterfactual test.** Off-policy, estimate what B2 *would* have achieved on this cohort. If it beats B0, the coverage minimum is set too conservatively and should move.

7.5 UC-5 Priya — four minutes to the interchange

The switching arm: `WINDOW_CLOSING`**. Direction: mostly refusal. The most important decision here is a switch not taken.**

User profile

- **Priya Raman**, 41. `age_cell` A5, Established professional, family with children.
- Home zone **IJburg**, work zone **Zuidas** — a thinly served origin, so her trips run to 3 legs with a last-mile hop.
- Device: **High-tier**. Battery 41%, thermal 0.
- Free text: *"I travel with my child sometimes, so family-friendly offers matter"*; *"can you only show me offers that don't require detours from my route?"*

LearnPerk engagement

- 31 offers seen, 7 accepted. Mean detour seen 3.1 minutes — well below the 6.2-minute dataset mean, because she filters hard on detour.
- Estimated detour tolerance ~2.4 minutes, in the bottom quartile. Detour is the dominant negative term in her acceptance.
- Content: low appetite (0.6), bounce rate 21% — she screens out almost everything.
- Tier Platinum, `points_balance` 1,240.

The moment. 08:41. She is on the second leg of a three-leg trip, approaching the interchange at Zuid with about four minutes of in-vehicle time left, then a 90-second platform transfer. The journey-state classifier has moved to `interchange-imminent`. She stands and starts walking towards the door, so `motion_state` becomes `Walking` and `ambient_location_precision` degrades to `Inferred`. The interchange is the highest-value moment the v3 document identifies for time-sensitive offers, and it is also the shortest.

Metric	Reading at the moment	Boundary	Verdict
W remaining window	240 s in-vehicle, ~90 s transfer	Latency budget tightens from 120 ms to 60 ms below 120 s	**WINDOW_CLOSING raised.** All κ charges roughly quintuple against a median leg
S journey stage	`interchange-imminent`	Selects the tight budget and the short-expiry offer filter	High value, low time
ρ location precision	`Inferred` (32.4% of impressions)	Not a gate; a discount on geographic models	**Discounts B3**, whose leading features are detour and merchant zone
κ for a B2 → B3 upgrade	~110 mJ load, amortised over 240 s = ~0.46 mJ/s	Against B3's utility gain, itself discounted by Inferred location	**Upgrade rejected.** Same models, same scores, opposite decision to UC-3 — and the only thing that changed is W
B battery / **N**	41%, Online	No pressure	Resources are not the issue here
Content appetite	Median content length 145 word-units; her bounce rate 21%	Below 60 s remaining, content is suppressed entirely	**Task C suspended** rather than downgraded

Table 7.9 — UC-5 readings. Compare row 4 directly against Table 7.5 row 7: identical models, identical switch cost, opposite outcome, because the window is 5.5× shorter.

Task	Switch	Trigger	Why this model, on these metrics
B offers	B2 → **B2** (hold); B3 upgrade **refused**	WINDOW_CLOSING + ρ	**The decision the case exists for.** B3 is feasible on latency — 9 ms p95 on high-tier, well inside a 60 ms budget — so a naive latency rule would have allowed it. The score refuses it on two grounds the rule cannot see: κ amortised over 240 s is five times its UC-3 value, and Inferred location discounts exactly the geographic features B3 exists to exploit. The incumbent is scored at κ = 0, so holding wins cleanly.
B candidate filter	Unchanged model, **restricted candidates**	Stage = interchange	Not a model switch but a constraint the stage imposes: only offers with `detour_minutes` under 2 and an expiry window long enough to survive the transfer are eligible. Her own free text asked for precisely this, and her 2.4-minute tolerance confirms it.
C content	C1 → **suspended**	W below threshold	A 145-word item cannot be read in the time available, and she screens out 21% of items in under two seconds at the best of times. Showing content here would spend energy to produce a guaranteed bounce. Suspension is a switching decision, and the floor model is *nothing*.

Task	Switch	Trigger	Why this model, on these metrics
E language	Slot **not loaded**; E0 only	κ over a small W	Loading E2 would cost ~340 ms and a large block of memory to serve a question that has about 240 s to arrive. E0 still answers the one question this stage actually produces — *"will I make my connection?"* — from structured timetable and delay facts, with no generation at all.
D incentive	D1 → **D1**, arms restricted	Stage	The posterior is re-scored over short-expiry arms only. A 24-hour offer is worthless to someone who will act in the next four minutes or not at all.

Table 7.10 — UC-5 switches. Only one task changes model. The engine's work here is refusing an upgrade, restricting candidates and suspending a surface.

Why this is efficient

This case is the clearest demonstration that κ belongs in the score rather than in a rule. UC-3 and UC-5 present the identical B2 → B3 decision with identical model costs, and the engine reaches opposite conclusions purely because W is 1,320 s in one and 240 s in the other. No threshold was tuned to produce that; it falls out of dividing the switch cost by the remaining window. The measurable saving is modest — one avoided model load, one avoided 200 MB language load, and a suspended content surface, together perhaps 450 mJ — but the avoided *waste* is total: every one of those spends would have produced output arriving after Priya had already left the vehicle. The definition of efficiency that matters at an interchange is not energy per inference, it is the share of inference that reaches a commuter still present to receive it.

How we test this case

- **Symmetry test.** Run UC-3 and UC-5 with every input identical except W. Assert opposite decisions on the B2 → B3 upgrade. This single test validates the entire κ-amortisation mechanism, and it is the one to write first.
- **Refusal-is-logged test.** Assert the refused B3 upgrade appears in `edge_switch_events` with `decision_type = hold` and both scores recorded. An unlogged non-decision is indistinguishable from a bug.
- **Suspension test.** Assert Task C returns an empty result below the window threshold and that this is recorded as a decision, not as a serve failure. The two must not be conflated, because one of them triggers a blacklist.
- **Precision-discount test.** Two identical impressions differing only in `ambient_location_precision`. Assert the geographic model scores lower under `Inferred`.
- **Outcome test.** Off-policy, restricted to interchange-stage impressions: does restricting to sub-2-minute detours raise acceptance? Her stated preference and her 2.4-minute estimated tolerance both predict yes, and the data can settle it.

7.6 Cross-case summary

Read together, the five cases show the engine moving in four different directions from one mechanism. The table below is the compressed argument of this whole document.

	UC-1 Lisa	UC-2 John	UC-3 Sanne	UC-4 Daan	UC-5 Priya
Trigger	CONNECTIVITY _LOSS	RESOURCE _PRESSURE	OPPORTUNITY	COVERAGE_LOSS	WINDOW _CLOSING
Direction	Down + sideways	Down, stepwise	**Up**	Sideways (both ways)	Refusal + suspend
Binding metric	N, then K and Φ	B vs personal threshold; ΔB; U	Δt, W, B, screen — all four	Φ and H	W, via κ
Mode	Balanced + M5	Frugal	**Opportunity**	Balanced	Balanced
Task A	A1 hold	A1 → A0	A1 → **A2**	A1 hold	A1 hold
Task B	B3 → B2	B2 → B1	B2 → **B3**; B4 rejected	B2 → B0 → B1	B2 hold; B3 **refused**
Task C	C2 → C1	C2 → C1	C2 hold	C2 → C0 → C1	**suspended**
Task D	D1 hold	D1 **hold**	D1 → **D2**	D1 → D0	D1, arms restricted
Task E	E4 → E0+E1	E1 → E0, unloaded	E1 → **E2 preloaded**	E0 → **E1**	not loaded
Primary gain	Availability: 11 minutes that are currently blank	Charge retained below a 24% disengagement threshold	Acceptance at the highest-value moment	Avoids a confident wrong answer to a new user	Avoids spend that would arrive after she has gone

*Table 7.11 — The five cases side by side. Three observations are worth drawing out. First, **Task D holds in UC-2 and Task B holds in UC-5**: the engine leaves things alone when leaving them alone is right, which a rule-based throttle would not. Second, **UC-3 moves up and UC-4 moves in both directions at once**, so the loop is not a one-way ratchet. Third, **the primary gain is different in every column** — availability, charge, acceptance, correctness, avoided waste — which is why the evaluation in §9 cannot be a single number.*

8 What this changes in the Data Dictionary

The v3 document anticipated that the data model would move once switching was specified. This section says exactly how. Nothing proposed here removes or alters an existing column; the changes are three new edge files, one new reference table, and a set of additive columns on the four files that already exist.

The organising principle is the one the current dictionary already follows: **one file per grain**. An inference and an impression cannot honestly share a row, because a single impression may be preceded by several inferences and a switch may occur with no impression at all. Forcing them together would produce exactly the dishonest join the dictionary warns against when it says content views and offer impressions each need their own file.

> **Why this data has to exist at all**
>
> Without it, every claim in §7 is unfalsifiable. We could not say whether a switch helped, whether a refused upgrade was right, whether M6 earns its cost, or whether the engine is simply a throttle that happens to be described in adaptive language. The current dictionary records what the commuter did; it does not record **which model decided it**, so today an acceptance and a refused are indistinguishable in the data. Adding `serving_model_id` to the impression files is the single highest-value change in this section, and it is also the cheapest.

8.1 Three new edge files

8.1.1 `edge_model_inference_log.csv` — one row per inference call

The finest grain in the system, and the only place per-call cost and confidence are recorded. High volume, so it is sampled: every call in the first 48 hours after install, every call under a triggered uncertainty, and a 5% sample otherwise.

Column	Type	What it is
`inference_id`	key	Primary key. One model call.
`user_id, leg_id, trip_id`	fk	Joins to every existing file at the grain those files already use.
`task_id`	enum	A, B, C, D or E.
`model_id`	fk	Joins to `model_registry`. The answer to “which model produced this?”
`invoked_at`	timestamp	Local Amsterdam time, matching every other file.
`latency_ms`	float	Wall clock around the call. The metric **L**.
`cpu_pct, mem_mb, energy_mj`	float	Metrics **U**, **R**, **E** at the moment of the call.
`confidence_top1, margin_top1_top2`	float	Metrics **C** and **M**. Calibrated, per the registry’s calibration version.
`n_candidates`	int	Size of the candidate set. A margin means nothing without it.
`feature_coverage`	float	Metric Φ, the realised value gate G4 compared against.
`features_missing`	string	Semicolon-separated names of the null required fields — the same array convention the dictionary already uses for `truth_taste_vector`.
`operating_mode`	enum	M1 to M6 at call time.
`connectivity_at_call, battery_at_call, thermal_status, cache_state_at_call`	mixed	Context snapshot. Duplicated from the leg deliberately, because leg-level values do not resolve a within-leg switch.
`delegate`	enum	`cpu`, `nnapi`, `gpu` or `npu`. Metric Γ as realised.
`was_cold_start`	bool	1 if the model was loaded for this call. Separates steady-state cost from load cost, which is what makes κ measurable rather than assumed.
`outcome_status`	enum	`served`, `suppressed`, `timeout`, `error`, `empty_candidates`. Feeds metric Ω. Five typed outcomes rather than one failure flag, following the `confirmation_status` precedent.
`impression_id, view_id`	fk, nullable	The downstream row this call produced, where it produced one. Null means suppressed or unsurfaced — meaningful, not zero.

Table 8.1 — `edge_model_inference_log.csv`. Roughly 18 columns. This is the file that makes model comparison possible at all.

8.1.2 `edge_switch_events.csv` — one row per switching decision

One row for every decision the planner reaches, including the decisions to **hold** and the switches that **rolled back**. Recording only the switches that happened would make UC-5 — whose central event is a refusal — invisible.

Column	Type	What it is
`switch_id`	key	Primary key.
`user_id, leg_id, task_id`	fk / enum	Context.
`decided_at`	timestamp	When the planner decided.
`from_model_id, to_model_id`	fk	Equal when `decision_type = hold`.
`decision_type`	enum	`exploit`, `explore`, `gate_forced`, `hold`, `rollback`, `mode_change`. Separating explore from exploit is what makes the ε-greedy fairness analysis possible at all.
`uncertainty_type`	enum	The typed trigger from Table 4.4, or null for a routine re-score.
`trigger_metric, trigger_value, boundary_value`	string, float, float	Which metric fired, what it read, what it was compared against. This triple is what makes a switch *explainable*, and the decision map lists explainability as a goal.
`score_from, score_to, ema_delta, margin_required`	float	Both sides of the comparison, plus δ. Lets us audit whether hysteresis is set well without re-running the device.
`kappa_charged`	float	The amortised switch cost applied to the challenger. The number that distinguishes UC-3 from UC-5.
`window_remaining_s`	float	Metric **W**, the κ denominator.
`mode_before, mode_after`	enum	M1 to M6.
`dwell_ms_before_switch`	int	How long the incumbent held the binding. Tests τ.
`switch_latency_ms, memory_delta_mb`	float	What the switch actually cost. EdgeMLBalancer reports 0.85 s mean switch time as a headline result; this is the column that lets us state ours.
`rollback_reason`	enum, nullable	`warmup_failed`, `warmup_too_slow`, `oom`, or null.

Table 8.2 — `edge_switch_events.csv`. Roughly 20 columns. Low volume: a handful of rows per leg.

8.1.3 `edge_adaptation_state.csv` — one row per 5-second tick

The time series the whole loop runs on, and the file that lets us replay a leg and ask why a switch did or did not happen. On-device it is a ring buffer holding the last 24 hours; it is uploaded in aggregate form on Wi-Fi, and in full only for commuters enrolled in the field trial of §9.6.

Column group	Columns	Notes
Keys	`tick_id, user_id, leg_id, observed_at`	Sorted by `observed_at`, like `event_stream.csv`.
Active bindings	`model_a, model_b, model_c, model_d, model_e`	One column per task rather than a JSON blob, so the file stays queryable without parsing.
Mode and control	`operating_mode, alpha, epsilon_current, theta_energy, theta_latency`	The live control state. `theta_energy` is the dynamic boundary of §4.3 — without logging it, a switch that a tightened boundary caused looks arbitrary.
Group R	`cpu_pct, mem_model_mb, battery_pct, battery_drain_rate, thermal_status, energy_mj_window, delegate_available`	Smoothed over the rolling window, except thermal, which is instantaneous because it gates.
Group Q	`ema_score_active, confidence_ema, margin_ema, coverage, drift_psi, serve_failures_window`	`drift_psi` is computed against the reference distribution shipped with the active model.
Group C	`connectivity_state, window_remaining_s, journey_stage, crowding_level, delay_minutes, cache_state, motion_state, screen_state, app_foreground_state, location_precision`	Names deliberately match the existing dictionary exactly, so a join needs no translation layer.
Raised	`uncertainties_raised`	Semicolon-separated typed uncertainties for this tick. Empty on most ticks, which is itself the useful signal.

Table 8.3 — `edge_adaptation_state.csv`. Roughly 35 columns at about 140 rows per median leg.

8.2 One new reference table

`model_registry.csv` is the static counterpart to the runtime files — the Knowledge component of §2.2 written down. It is versioned with the app, and every `model_id` in the three files above joins to it.

Column	What it is
`model_id, task_id, tier, family, version`	Identity. `family` is e.g. `lightgbm`, `logreg`, `two_tower`, `slm`, `rule`.
`display_name, description`	The human name, and the natural-language task description CALM-style routing computes cosine similarity against. For Task E this column is not documentation — it is an input.
`file_size_mb, peak_rss_mb, quantisation, delegate_required`	Footprint. Feeds gates G1 and G6.
`p50_ms_low, p95_ms_low, p50_ms_mid, p95_ms_mid, p50_ms_high, p95_ms_high`	Measured latency by device class. Six columns rather than one, because a single figure is meaningless across a 30/45/25 fleet split.
`energy_mj_per_call_low / _mid / _high`	Measured energy by device class, calibrated once per tier.
`offline_capable, min_feature_set, min_coverage, expected_null_fields`	Feeds gates G3 and G4. `expected_null_fields` is the declaration that makes rule 3 of §6.2 enforceable.
`switch_energy_mj_from`	Load-and-warm cost, keyed by the outgoing model. The empirical basis for κ.
`trained_on_version, reference_distribution_ref, calibration_version, ece_at_release`	Provenance. `reference_distribution_ref` points at the distribution the drift test compares against, which is HarmonE's versioned-repository idea reduced to one column.
`regime`	`general`, `peak`, `offpeak` or `disrupted`. How the DRIFT tactic finds a sibling to switch to instead of retraining.
`min_tier_permitted_thermal, min_device_class`	Hard eligibility, feeding gate G2.

Table 8.4 — `model_registry.csv`, about 30 columns and as many rows as there are models — 20 under the spectrum proposed in §5.

8.3 New columns on the four existing files

All additive. No existing column changes meaning, and no existing null becomes a zero.

File	New columns	Why
`combined_offers.csv` 9,990 rows, 92 → 100 columns	`serving_model_id, serving_model_tier, model_confidence, inference_latency_ms, operating_mode_at_impression, feature_coverage_at_impression, switch_within_leg_flag, fallback_reason`	Without `serving_model_id` the acceptance target cannot be attributed to a model, and every comparison in §9 is impossible. `fallback_reason` is nullable and populated only when the gate forced a model that was not the scored best — a null here means the engine got its first choice.
`combined_content_views.csv` 19,334 rows, 43 → 48 columns	`serving_model_id, model_confidence, inference_latency_ms, operating_mode_at_view, from_cached_pool`	`from_cached_pool` is the column that will let us measure whether the offline content path of UC-1 works. Today the file has zero offline rows, so this column starts at zero by construction and its growth *is* the result.
`combined_users.csv` 2,000 rows, 49 → 58 columns	`n_switches_total, n_rollbacks, pct_time_tier0, pct_time_tier3plus, modal_operating_mode, mean_inference_latency_ms, est_energy_mj_total, personal_battery_threshold, model_selection_entropy`	Per-commuter summaries in the style the file already uses. `model_selection_entropy` is the fairness measure: EdgeMLBalancer reports selection disparity as a headline result, and entropy over the selection distribution is the per-user version of it. `personal_battery_threshold` is the learned value UC-2 turns on.
`event_stream.csv` 70,758 rows, 7 columns unchanged	No new columns. Four new `event_type` values: `model_switch`, `mode_change`, `drift_alarm`, `serve_failure`.	The existing schema already carries these cleanly: `subject` takes the task, `value` takes the trigger reading, and `detail` takes the uncertainty type. Replaying a commuter's history in order then includes their switching history, which is exactly what the file is for.

Table 8.5 — Additive changes to the four existing files. Twenty-two new columns in total, and four new event types.

8.4 Retention, consent and privacy

Four constraints, and they are constraints rather than guidance.

1. **Adaptation telemetry is device state, not behaviour.** CPU, memory, thermal and latency describe the phone, not the person, and are aggregated before upload. Group C metrics are different: journey stage and crowding are behavioural, and they inherit the same consent gate as every other behavioural row.
2. **Consent withdrawal removes switching rows too.** The dictionary is unambiguous that an opted-out commuter has *no behavioural rows anywhere in the dataset, not blank ones*, and that no event exists after `withdrawal_at`. The three new files follow the same rule without exception.
3. **Retention is tiered.** `edge_adaptation_state` is a 24-hour on-device ring buffer, uploaded in aggregate; `edge_model_inference_log` is sampled and kept 30 days; `edge_switch_events` is low-volume and kept 90 days, because the switching analysis needs a longer baseline than the raw telemetry does.
4. **Nothing here is a new personal data category.** Every Group C field already exists in the dictionary. What is new is the record of *which model read them*, which is metadata about our system rather than additional data about the commuter — and it is what makes the explainability goal in the decision map deliverable rather than aspirational.

9 How we test the switching

A switching engine is easy to build and hard to justify. It will always produce switches, and the switches will always look purposeful in a log. The question this section answers is how we establish that the engine is **better than not having one**, and better than the much simpler things we could have built instead.

The evaluation has three stages, in increasing cost and decreasing control: offline replay on the data we already hold, on-device measurement on a fixed workload, and a field trial. Each stage can kill the design before we pay for the next one.

9.1 Research questions

RQ	Question	Stage	Kill condition
RQ1	Does switching retain acceptance? Does the engine stay within 2 percentage points of the always-heaviest configuration on offer acceptance?	Replay, then field	A loss above 2 pp that a weight change cannot close.
RQ2	Does it cost less? How do energy, p95 latency and peak memory compare against each fixed-model baseline and against the naive and round-robin policies?	On-device	No significant energy reduction against the always-heavy baseline.
RQ3	Does it close the availability gap? What share of offline legs receive a serveable recommendation, against a present-day baseline of zero content views offline?	Replay, then field	Below 95% of offline legs served.
RQ4	Is the selection fair? Does ε-greedy avoid the starvation EdgeMLBalancer measured — one model taking 74.3% of frames under naive, 84.4% under round-robin?	Replay	Any single model above 60% of selections within a task, sustained.
RQ5	Is switching itself cheap? What are mean switch latency, switch frequency and rollback rate, and what fraction of a leg is spent mid-switch?	On-device	Above 3% of leg time mid-switch, or a rollback rate above 2%.
RQ6	Does the opportunity mode earn its cost? Restricted to impressions where M6 holds, is the acceptance lift of the heavier model positive?	Replay, off-policy	A non-positive lift. M6 would then be retired, not tuned.
RQ7	Which mechanisms matter? An ablation over κ, the coverage gate, the EMA, the dynamic boundary, exploration and the semantic term.	Replay	Any mechanism with no measurable effect is removed from the design.

Table 9.1 — Seven research questions, each with a stated kill condition. RQ1 to RQ5 mirror the questions EdgeMLBalancer, HarmonE and CALM each ask of themselves; RQ6 and RQ7 are ours.

9.2 Baselines

Nine experimental candidates, following HarmonE's practice of comparing against both every fixed model and every simpler adaptive policy. Comparing only against the heaviest model would flatter us; comparing only against the lightest would flatter us differently.

Candidate	What it is	What it isolates
Fixed-B0 … Fixed-B4	Five runs, one per tier, no switching at all.	The accuracy/cost curve the spectrum spans. Every other result is read against this curve.
Naive-Threshold	Fixed CPU cut-offs at 20/25/30%, exactly the policy in the EdgeMLBalancer reference implementation's naive variant.	Whether the composite score beats a threshold table. EdgeMLBalancer found naive gave one model 74.3% of frames and the lowest accuracy of the three policies tested.
RoundRobin+Boost	Time-sliced rotation with periodic CPU recalibration, the EdgeMLBalancer MLFQ variant.	Whether context-aware selection beats fair-by-construction rotation. Their result: better accuracy than naive, but still 84.4% concentration and the slowest switching at 1.40 s.
Switch-NoGoals	Our engine with the decision map removed: score only, no boundaries, no modes.	Whether HarmonE's design-time goals contribute, or whether the score alone suffices.
LearnPerk-Balancer	The full design in this document.	The proposal.

Table 9.2 — Nine candidates. Note that two are lifted directly from the EdgeMLBalancer repository, so the comparison is against running code rather than a described policy.

9.3 Replay harness and off-policy evaluation

The replay harness reads `event_stream.csv` in order — which is what the dictionary says the file is for — and reconstructs each leg as a sequence of 5-second ticks, synthesising the Group R metrics from a device model calibrated against real hardware and taking every Group C metric straight from the file. The engine then runs unmodified against that tick stream, and we compare the model it selects against the model each baseline would have selected.

Measuring the *consequence* of a different selection is the harder half, because the logs record what happened under the policy that actually ran. The dictionary anticipates this precisely: `propensity` is described as "the probability this particular offer was chosen to be shown — needed for any later off-policy evaluation of a ranking change; without it that analysis is impossible." That column is what makes RQ1 and RQ6 answerable at all.

1. **Inverse propensity scoring, self-normalised (SNIPS)**, weighting each logged impression by the ratio of the new policy's selection probability to `propensity`. Self-normalised rather than plain IPS, because the variance of plain IPS at these sample sizes would swamp a 2 pp effect.
2. **A doubly robust estimator** as a cross-check, using a separately fitted outcome model. Agreement between the two is the evidence that the estimate is trustworthy; disagreement is a reason to stop and look rather than a reason to pick the friendlier number.
3. **The `truth_` columns as an upper bound, never as features.** `truth_p_accept` is documented as the theoretical ceiling any model can reach. Reporting each candidate as a fraction of that ceiling is far more honest than reporting a raw accept rate, and it is only possible because the simulator kept the answer key.
4. **Effective sample size reported alongside every estimate.** With 9,990 impressions and a propensity floor near 0.03, some comparisons will simply not have the support to resolve a 2 pp difference, and we should say so rather than report a number.

> **The honest limit of replay**
>
> Replay can tell us which model the engine *would* select, and off-policy estimation can tell us roughly what that selection would have been worth. It cannot tell us the energy cost, because the logs contain no energy measurement, and it cannot tell us the switch latency, because no switch ever occurred. RQ2 and RQ5 are therefore **not answerable offline at all** and must wait for stage two. Claiming an energy result from replay would be the single easiest way to make this work look better than it is.

9.4 Evaluation metrics

Family	Metric	How measured	Target
Quality	Offer acceptance rate; fraction of the `truth_p_accept` ceiling attained	SNIPS over replay; direct measurement in the field	Within 2 pp of always-heavy
Quality	Content completion rate and bounce rate	Direct	No worse than the 21.5% completion and 10.7% bounce currently observed
Availability	Share of offline legs with a serveable recommendation	Direct	≥ 95%, against a present-day content baseline of **0%**
Cost	mAh per commuting hour attributable to LearnPerk	On-device, matched 30-minute runs with 30-minute cooldowns	Below the 18 mAh/hr decision-map boundary
Cost	p95 surface latency by journey stage	On-device	≤ 120 ms normal, ≤ 60 ms when W < 120 s
Cost	Peak model-attributable RSS	On-device	≤ 160 / 280 / 900 MB by device class
Switching	Mean switch latency; switches per leg; fraction of leg spent mid-switch	From `edge_switch_events`	Comparable to EdgeMLBalancer's 0.85 s; under 3% of leg time
Switching	Rollback rate	From `rollback_reason`	< 2% of attempted switches
Fairness	Selection distribution per task; entropy; max single-model share	From `edge_switch_events`	No model above 60% sustained — against 74.3% and 84.4% for the two baselines
Robustness	Serve-failure rate; ticks with an empty feasible set	Direct	Serve failures < 0.5%; empty feasible set never, since the floor model must always pass

Table 9.3 — Evaluation metrics. Ten metrics across five families, because as Table 7.11 shows, the gain differs by case and a single headline number would hide four of the five.

9.5 Per-use-case acceptance tests

The tests written into each case in §7 form the acceptance suite. Collected here with their priority: these run on every build, and a failure blocks release.

Priority	Test	From	What a failure would mean
P0	Symmetry: UC-3 and UC-5 differ only in W and reach opposite conclusions on the same upgrade	UC-5	κ amortisation is not working, which removes the central mechanism of the design. Write this test first.
P0	Floor-model guarantee: the feasible set is never empty for any task under any combination of gates	§4.1	The engine can go dark, which fails the decision map's hardest goal.
P0	Null handling: no null is ever imputed as zero anywhere in the feature pipeline	UC-4	Three distinct facts collapse into one, exactly as the dictionary warns.
P1	Gate immediacy: connectivity loss fires on the same tick, bypassing dwell	UC-1	Serving failures for the length of the smoothing window.
P1	Down-switch precision: the engine picks the best feasible model, not the cheapest	UC-1	Accuracy given away for no resource benefit — the most likely subtle bug.
P1	Non-switch: D1 survives ten ticks in Frugal mode	UC-2	A blanket throttle wearing the vocabulary of an adaptation loop.
P1	Personal threshold: two commuters at identical charge behave differently	UC-2	A global constant where the data says the threshold is personal.
P1	Composite trigger: M6 requires all four conditions; three negative cases pass	UC-3	Expensive models running when the moment does not justify them.
P2	Refusal logging: held and rejected candidates appear with both scores	UC-5	Non-decisions become invisible and the engine becomes unauditable.
P2	Hysteresis: oscillating a metric across a boundary produces at most one switch	UC-2	Thrash, which at 5-second planning would be continuous.
P2	Direction independence: one trigger moves one task down and another up	UC-4	Coverage has been conflated with cost.
P2	Accessibility invariance: rendering is identical across all tiers	UC-2	Accessibility became a casualty of a down-switch, which is not an acceptable trade at any energy saving.

Table 9.4 — Twelve acceptance tests. Three are P0 and block release outright.

9.6 On-device field trial

Stage three, and the only stage that can answer RQ2 and RQ5. The protocol follows EdgeMLBalancer's, because their threats-to-validity section identifies the right controls: device reset before each run, full charge at the start, a warm-up phase to stabilise the hardware, and a cooldown between runs to prevent carryover.

Aspect	Design
Devices	Three per class, nine total, spanning the 30/45/25 low/mid/high split. Each class runs the full candidate set, because a result on one class does not transfer — that is the whole reason the registry carries six latency columns.
Workload	One recorded 30-minute commute trace, replayed identically to every candidate, so the comparison is like-for-like. EdgeMLBalancer used a fixed 30-minute traffic recording for exactly this reason.
Repetition	Five independent runs per candidate per device class, with a 20-minute cooldown between consecutive runs, following HarmonE's protocol and standard empirical software engineering guidance.
Controls	Device reset and full charge before each run; a warm-up phase; airplane mode toggled on a fixed schedule to reproduce the tunnel; background apps cleared.
Induced conditions	Connectivity loss on a schedule matching the measured 22.9% offline and 35.6% degraded metro profile; battery drained to target levels before low-charge runs; thermal load induced by a controlled background task.
Drift induction	Following HarmonE, a scale-and-shift transformation applied twice to a segment of the feature stream, so the distribution first diverges and later realigns. This tests both that drift is detected and that the regime-specific sibling is *reused* rather than a retrain being triggered.
Shadow phase	Before any commuter-visible rollout, the engine runs in shadow: it selects and logs but does not bind, while the production model continues to serve. Gives a full `edge_switch_events` history at zero user risk, and catches a mis-specified gate before anyone experiences it.
Staged rollout	1% of commuters, then 5%, then 25%, with the RQ1 acceptance boundary as the automatic stop condition at each gate.

Table 9.5 — Field trial protocol. The shadow phase is the most important row: it produces the switching data this design needs without exposing a single commuter to a switching bug.

10 Threats to validity and open questions

Stated in the form the three source papers use, because the same categories apply and because a design document that lists no threats is not being honest about its confidence.

Construct validity — do the metrics measure what we claim?

- **Energy is estimated, not measured.** There is no pyRAPL on Android. HarmonE could profile energy directly at the system level; we infer it from CPU time times a per-device-class power coefficient. The coefficient will be wrong under thermal throttling, which is precisely when it matters most. Mitigation: calibrate per device class against a hardware power monitor, and treat **E** as a gate input only in combination with the directly measured Δ**B**. The EdgeMLBalancer reference code computes battery consumption as `battery_level × cpu_usage / 100`, which multiplies a level by a rate and is not a quantity of energy at all; we should not reproduce that.
- **Confidence is a proxy for correctness, and a weak one.** Gate G4 exists because a model fed nulls returns arbitrary answers *confidently*. The same failure can occur inside the coverage minimum. Mitigation: ECE is tracked per model and a badly calibrated model is demoted in the registry, but calibration is recomputed nightly, so there is a window in which the switching engine is optimising a number that has stopped meaning anything.
- **The remaining window W is itself a prediction.** It comes from `duration_minutes` corrected by live delay, and it is the denominator of κ — so an error in W propagates into every switch-cost decision. On a 4-minute window a 60-second error is a 25% error in κ. Mitigation: floor W at `W_min` so the charge cannot diverge, and report W prediction error as a first-class result rather than an implementation detail.

Internal validity — could something else explain the result?

- **The dataset is simulated.** The dictionary is explicit that the `truth_` columns are the traits the simulator used to drive behaviour. That is a genuine advantage for off-policy evaluation, since it gives a known ceiling, but it means an engine tuned against replay is tuned against a generator. The relationships a model learns are the ones that were put in. Mitigation: replay is a *screening* stage with kill conditions, never a proof; RQ2 and RQ5 are excluded from it entirely.
- **Device thermal and background state confound everything.** A heavier model is slower partly because it is heavier and partly because it heated the phone on the previous run. Mitigation: EdgeMLBalancer's controls — reset, full charge, warm-up, cooldown — adopted verbatim in §9.6.
- **The engine changes the data it learns from.** Switching alters which impressions occur, which alters the rolling metrics, which alters future switching. This is the standard feedback loop in any deployed ranker and it is not solved here. Mitigation: `propensity` is logged for every selection, including model selection and not only offer selection, so the loop remains analysable after the fact.

External validity — how far does this generalise?

- **Amsterdam, GVB, and the metro in particular.** The strongest argument in §1 — that only 41.6% of metro impressions are online — is a fact about tunnels under the IJ. A surface-rail network in a different city would weaken the connectivity case considerably, though the battery, window and coverage arguments would survive untouched.
- **Android only.** Every probe named in §3.1 — `/proc/self/stat`, `getCurrentThermalStatus()`, NNAPI — is Android-specific, as EdgeMLBalancer's was. iOS would need an equivalent monitor layer, and iOS exposes less.
- **The model sizes and latencies are targets, not measurements.** Every number in §5 is drawn from published figures for these architectures at these quantisations. They are good enough to reason about the spectrum and not good enough to commit to. The registry must be built and measured before any of §7's efficiency numbers can be stated as fact rather than as expectation.

Open questions we cannot settle from this document

Question	Why it is open	How we would resolve it
Should Task E exist on low-tier devices at all?	E1 at ~23 MB is 14% of a 160 MB ceiling for intent classification that a phrase table handles adequately most of the time.	Measure E0's accuracy against E1's on real GVB bulletins. If the gap is small, low-tier ships E0 only and the ceiling problem disappears.

Question	Why it is open	How we would resolve it
Is ε-greedy the right exploration mechanism here?	EdgeMLBalancer’s fairness argument is about preventing starvation of models. Our spectrum has a genuine ordering, so exploring into B0 when B3 is feasible may just be a worse answer rather than a useful sample.	Ablate against Thompson sampling over the score posterior, and against no exploration at all, in RQ7.
Should the operating mode be per-task rather than global?	Task E dominates the memory budget, so a single global mode may unnecessarily downgrade the cheap tasks whenever the language layer is the actual problem.	Run a per-task-mode variant in replay and compare acceptance retention at equal energy.
How is the per-user battery threshold learned before we have observed disengagement?	It is circular: we need past disengagement to set the threshold, and the threshold is meant to prevent disengagement.	Start at the population median of the documented 20–40% band and update with a hierarchical prior as evidence accrues. State explicitly that UC-2 does not work for a new user.
Does the cloud tier belong in the spectrum at all?	E4 is gated out on 22.9% of metro impressions and raises consent questions that E0 to E3 do not. It may be simpler to remove it than to gate it.	Measure how often E4 is both feasible and selected. If that is a small share of questions, the complexity is not earning its place.

Table 10.1 — Five open questions. Each is a decision this document deliberately does not make.

11 Appendix — parameter defaults

Every tunable in one place. These are starting points chosen to be defensible, not tuned values; the RQ7 ablation in §9.1 exists to replace them.

Symbol	Parameter	Default	Source
—	Monitor sample period	1 s	EdgeMLBalancer reference implementation
—	Plan period	5 s	This document (§2.3)
`n`	Rolling window	12 ticks (60 s)	This document
`beta`	EMA smoothing factor	0.30	HarmonE
`gamma`	Dynamic boundary decay	0.20	HarmonE
`epsilon`	Exploration rate	0.10, annealed to 0.02 after 500 calls	EdgeMLBalancer (0.10); annealing is ours
`tau`	Minimum dwell time	20 s (A–D), 120 s (E)	This document
`delta`	Switch margin	0.05	This document
`alpha`	Quality/cost balance	0.75 / 0.55 / 0.35 / 0.15 / 0.85 by mode	HarmonE's role; our values
`lambda`	Semantic similarity weight	0.30, Task E only	CALM (reports k in 0.1–0.3)
`W_min`	Window floor for κ	60 s	This document
`PSI`	Drift threshold	0.25 on a top-10 feature	Conventional PSI practice; HarmonE uses KL
—	Blacklist duration after two serve failures	90 s	This document
—	M6 cap	300 s, no immediate re-entry	This document
—	Language cache size	1 resident model	CALM (cache_size = 1 configuration)
—	Latency budget	120 ms normal; 60 ms when W < 120 s	Decision map (§2.1)
—	Memory ceiling	160 / 280 / 900 MB by device class	Decision map
—	Energy reference level	18 mAh per commuting hour	Decision map

Table 11.1 — Parameter defaults and their provenance. Eight are inherited from the three source approaches; the rest are ours and carry no empirical backing yet.

Sources

[1]	A. Matathammal, K. Gupta, L. Lavanya, A. V. Halgatti, P. Gupta, K. Vaidhyanathan. *EdgeMLBalancer: A Self-Adaptive Approach for Dynamic Model Switching on Resource-Constrained Edge Devices.* arXiv:2502.06493v1 [cs.SE], 10 February 2025. Reference implementation: `github.com/sa4s-serc/EdgeMLBalancer` — the epsilon-greedy, naive and MLFQ CameraFragment variants were read directly for this document.
[2]	H. Bhatt, S. Biswas, S. Rakhunathan, K. Vaidhyanathan. *HarmonE: A Self-Adaptive Approach to Architecting Sustainable MLOps.* arXiv:2505.13693v1 [cs.SE], 19 May 2025.
[3]	H. Jain, D. Pandey, K. Vaidhyanathan. *CALM: A Self-Adaptive Orchestration Approach for QoS-Aware Routing in Small Language Model based Systems.* arXiv:2602.03632v1 [cs.SE], 3 February 2026.
[4]	*LearnPerk Edge Data & Model Switching*, v3, August 2026. Internal. The document this one was named as the follow-on to.
[5]	*LearnPerk Data Dictionary*, September 2026, generated from the data itself. Internal. All measured figures in §1.1 were computed directly from the four combined files it documents.
[6]	J. O. Kephart and D. M. Chess. *The Vision of Autonomic Computing.* IEEE Computer, 36(1), 2003. The origin of the MAPE-K loop all three approaches build on.
[7]	A. Marda, S. S. Kulkarni, K. Vaidhyanathan. *SWITCH: An Exemplar for Evaluating Self-Adaptive ML-Enabled Systems.* arXiv:2402.06351, 2024. The QoS-aware model-switching baseline CALM adapts and compares against.
[8]	M. Tedla, S. S. Kulkarni, K. Vaidhyanathan. *EcoMLS: A Self-Adaptation Approach for Architecting Green ML-Enabled Systems.* arXiv:2404.11411, 2024. The source of the performance-score form EdgeMLBalancer adapts.

Next steps, in order. (1) Build `model_registry.csv` and measure every latency, size and energy figure in §5 on real devices across all three classes — until that exists, every efficiency claim in §7 is an expectation rather than a result. (2) Implement the three new edge files of §8 and run the engine in shadow mode, which yields a complete switching history at zero user risk. (3) Run the replay harness and the RQ7 ablation to remove any mechanism that turns out not to matter. (4) Only then run the on-device trial that can answer RQ2 and RQ5.